



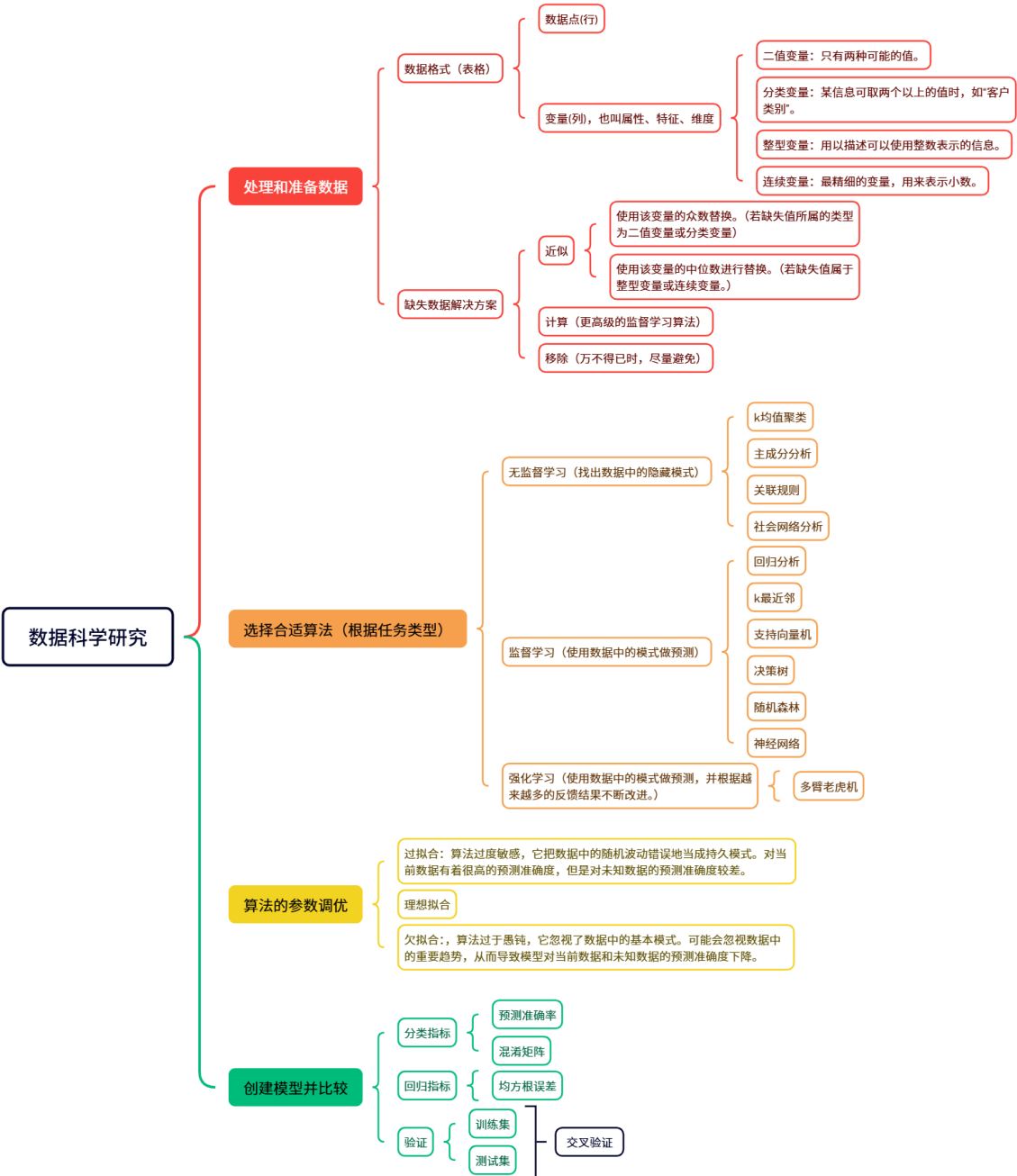
数据挖掘基本认识 课后作业1

 姓名：张苏芯

 班级：信管2301

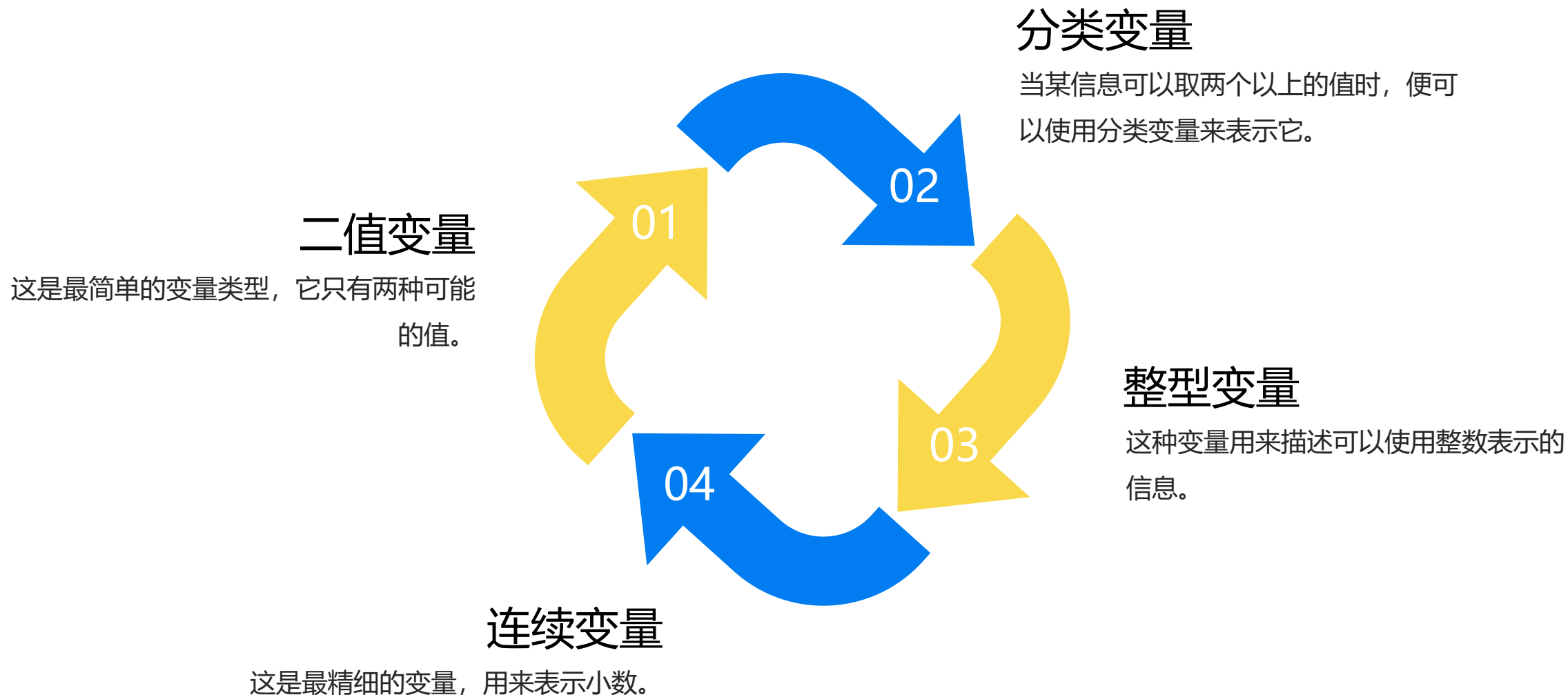
 学号：202321054007

阅读《白话机器学习算法》第1章，整理读书笔记。



阅读《白话机器学习算法》第1章，整理读书笔记。

1、准备数据——变量类型



阅读《白话机器学习算法》第1章，整理读书笔记。

缺失数据的处理

我们所收集的数据并非总是完整的，因此要尽可能地使用如下一些方法来解决数据缺失问题。



近似：如果缺失值所属的类型是二值变量或分类变量，那么可以使用该变量的众数来替换它。

若缺失值属于整型变量或连续变量则可以使用中位数替换它。



计算：对于缺失值，还可以使用更高级的机器学习算法将它计算出来。



移除：万不得已时可以把包含缺失值的整行数据移除，但是尽量不要这么做，因为这将会减少分析时可用的数据量。

阅读《白话机器学习算法》第1章，整理读书笔记。

2、选择算法



无监督学习

目标：找出数据中的隐藏模式；

算法：K均值聚类、主成分分析、关联规则、社会网络分析；



监督学习

目标：使用数据中的模式做预测；

算法：回归分析、K最近邻、支持向量机、决策树、随机森林、神经网络；



强化学习

目标：使用数据中的模式做预测，并根据越来越多的反馈结果不断改进。

算法：多臂老虎机

阅读《白话机器学习算法》第1章，整理读书笔记。

3、参数调优



过拟合：算法过度敏感，它把数据中的随机波动错误地当成持久模式。对当前数据有着很高的预测准确度，但是对未知数据的预测准确度较差。



理想拟合：算法能够较好地解释当前数据，并实现准确的预测。



欠拟合：算法过于愚钝，它忽视了数据中的基本模式。可能会忽视数据中的重要趋势，从而导致模型对当前数据和未知数据的预测准确度下降。

阅读《白话机器学习算法》第1章，整理读书笔记。

4、评价模型



分类指标

预测准确率：正确的预测所占的比例；

混淆矩阵：可以进一步了解预测模型的优缺点。



回归指标

均方根误差：可用于避免较大的误差，因为每个误差都取了平方，所以大误差就被放大了，对异常值极其敏感。



验证

训练集：用来生成和调整预测模型；

测试集：用来充当新数据并评估模型的预测准确度。

本章概念汇总

1. 大数据 (Big Data) : 又称海量数据, 指以不同形式存在于数据库、网络等媒介上蕴含丰富信息的规模巨大的数据。
2. 数据挖掘 (Data Mining) : 又称数据开采、数据采掘等, 是从大量的、不完全的、有噪声的、模糊的、随机的实际应用数据中, 提取隐含在其中的、人们事先不知道的, 但又是潜在有用的信息和知识的过程。
3. 数据降维 (主成分分析PCA、t-SNE) : 由于大数据体量巨大可能会超过计算机处理能力, 因此对数据进行缩减数据量的过程, 该过程可以提取最有用的信息, 从而获得更精简的变量集, 以供进一步分析。其要点是根据数据挖掘的目标、数据量、计算机的处理能力、对时间的要求等多方面因素对数据进行分级降维, 包括抽样和分散的降维方式。
4. ①有监督 (supervised) : 使用数据中的模式进行预测, 当需要做预测时, 就会用到回归分析、k最近邻、支持向量机、决策树、随机森林、神经网络等监督学习算法。
②无监督 (unsupervised) : 由于不知道要找的模式是什么, 而是要依靠算法从数据集中发现模式, 因此当希望找出数据集中隐藏的模式时, 可以使用k均值聚类、主成分分析、关联规则、社会网络分析等无监督学习算法。
5. ①结构化数据 (Structured Data) : 简单来说就是数据库。企业ERP、财务系统、医疗HIS数据库等。这类数据基本包括高速存储应用需求、数据备份需求、数据共享需求以及数据容灾需求。
②半结构化数据 (semi-Structured Data) : 包括邮件、HTML、报表、资源库等, 如邮件系统、WEB集群、教学资源库、数据挖掘系统、档案系统等, 这些应用对应于数据存储、数据备份、数据共享以及数据归档等基本存储需求。
③非结构化数据 (Non-Structured Data) : 包括视频、音频、图片、图像、文档、文本等形式。如医疗影像系统、教育视频点播、视频监控等具体应用, 这些行业对于存储需求包括数据存储、数据备份以及数据共享等。

本章概念汇总

6. ①数据预处理:数据准备的环节之一, 在现实世界中, 由于各种原因导致数据总是有这样或那样的问题, 从而导致数据质量的三个要素:准确性、完整性和一致性都很差。因此需要根据数据质量的不同采用不同的技术对数据进行处理的过程, 以得到准确性、完整性和一致性较好的数据。通常包括数据清洗、数据集成、数据归约和数据变换四个步骤。

②数据清洗:数据预处理的环节之一, 是通过填写缺失的值、光滑噪声数据、识别或删除离群点, 并解决不一致性等方式来“清理”数据。

7. ①数据标准化 (Normalization): 是将数据按比例缩放, 使之落入一个小的特定区间。在某些比较和评价的指标处理中经常会用到, 去除数据的单位限制, 将其转化为无量纲的纯数值, 便于不同单位或量级的指标能够进行比较和加权。

②数据归一化:是对原始数据的线性变换, 使结果落到 $[0, 1]$ 区间。

8. 衍生变量 (derived variable):是由其他既有变量通过不同形式的组合而衍生出的变量。例如, 已经知道一个长形物体的质量、长度、体积, 就可以通过对现有三个变量的组合得到一些有用的衍生变量, 如密度=质量/体积, 线密度=质量/长度。

回归算法 课后作业2

 姓名：张苏芯

 班级：信管2301

 学号：202321054007

1、整理数据回归方法的基本方法、步骤（J p125）



1、整理数据回归方法的基本方法、步骤 (J p125)

当人们对研究对象的内在特性和各因素间的关系有比较充分的认识时，一般用机理分析方法建立数学模型。如果由于客观事物内部规律的复杂性及人们认识程度的限制，无法分析实际对象内在的因果关系，建立合乎机理规律的数学模型，那么通常的办法是搜集大量数据，基于对数据的统计分析去建立模型。数据挖掘正是处理数据的技术，本章将讨论数据挖掘中用途非常广泛的一类方法——回归方法。

为什么要进行回归分析？

① 事物之间的关系可以抽象为变量之间的关系。② 变量之间的关系可以分为两类：一类叫确定性关系，也叫函数关系，其特征是：一个变量随着其他变量的确定而确定。另一类关系叫相关关系，变量之间的关系很难用一种精确的方法表示出来。例如，通常人的年龄越大血压越高，但人的年龄和血压之间没有确定的数量关系，人的年龄和血压之间的关系就是相关关系。④ 回归方法就是处理变量之间的相关关系的一种数学方法。其解决问题的大致方法、步骤如下：⑤ 回归分析的意义在于更加有效的描述、解释和预测事物(变量)之间的关系。

回归分析的具体步骤

- 1) 收集一组包含因变量和自变量的数据；
- 2) 选定因变量和自变量之间的模型，即一个数学式子，利用数据按照一定准则（如最小二乘）计算模型中的系数；
- 3) 利用统计分析方法对不同的模型进行比较，找出效果最好的模型；
- 4) 判断得到的模型是否适合于这组数据；
- 5) 利用模型对因变量作出预测或解释。

回归在数据挖掘中是最为基础的方法，也是应用领域和应用场景最多的方法，只要是量化型问题，我们一般都会先尝试用回归方法来研究或分析。

根据回归方法中因变量的个数和回归函数的类型（线性或非线性）可将回归方法分为以

2、整理一元线性回归 (J p126) 的基本数学模型;

下几种：一元线性、一元非线性、多元线性、多元非线性。另外还有两种特殊的回归方式，一种在回归过程中可以调整变量数的回归方法，称为逐步回归，另一种是以指数结构函数作为回归模型的回归方法，称为 Logistic 回归。本章将逐一介绍这几个回归方法。

7.1 一元回归

7.1.1 一元线性回归

设 Y 是一个可观测的随机变量，它受到一个非随机变量因素 x 和随机误差 ε 影响。若 Y 与 x 有如下线性关系：

数学模型

$$Y = \beta_0 + \beta_1 x + \varepsilon$$

且 ε 的均值 $E(\varepsilon) = 0$ ，方差 $\text{var}(\varepsilon) = \sigma^2 (\sigma > 0)$ ，其中 β_0 、 β_1 是固定的未知参数，称为回归系数， Y 称为因变量， x 称为自变量，则称此 Y 与 x 之间的函数关系表达式为一元线性回归模型。

对于实际问题，要建立回归方程，首先要确定能否建立线性回归模型，其次确定如何对模型中的未知参数 β_0 、 β_1 进行评估。

通常，我们首先对总体 (x, Y) 进行 n 次独立观测，获得 n 组数据（称为样本观测值）：

$$(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$$

然后在直角坐标系 xoy 中画出数据点 $(x_i, y_i), (i=1, 2, \dots, n)$ ，该图形称为数据的散点图。如果这些点大致地位于同一条直线的附近，或者说，散点图呈现线性形状，则认为 Y 与 x 之间的关系符合线性关系。此时，利用最小乘法可以得到回归模型参数 β_0 、 β_1 的最小二乘估计 $\hat{\beta}_0$ 、 $\hat{\beta}_1$ ，估计公式为：

$$\hat{\beta}_0 = \bar{y} - \bar{x} \hat{\beta}_1$$

$$\hat{\beta}_1 = \frac{L_{xy}}{L_{xx}}$$

$$\text{其中, } \bar{x} = \frac{1}{n} \sum_{i=1}^n x_i, \bar{y} = \frac{1}{n} \sum_{i=1}^n y_i, L_{xx} = \sum_{i=1}^n (x_i - \bar{x})^2, L_{xy} = \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y}).$$

于是就可以建立经验模型：

$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x$$

对于得到的回归方程形式，通常需要进行回归效果的评价，当有几种回归结果后，还需要加以比较以选出较好的方程，常用的准则有：

1) 决定系数 R^2 ，其数学定义为：

$$R^2 = 1 - \frac{SSE}{SST}$$

$$SSE = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$
$$SST = \sum_{i=1}^n (y_i - \bar{y})^2$$

R^2 称为决定系数。显然 $R^2 \leq 1$ ， R^2 大表示观测值 y_i 与拟合值 $\hat{y}_i$ 比较靠近，也就意味着

从整体上看， n 个点的散布离曲线较近。因此选 R^2 大的方程为好。

2) 剩余标准差 s ，其数学定义为：

$$s = \sqrt{SSE / (n-2)}$$

s 称为剩余标准差， s 类似于一元线性回归方程中对 σ 的估计，可以将 s 看成是平均残差平方和的算术根，自然其值小的方程为好。

其实上面两个准则所选方程总是一致的，因为 s 小必有残差平方和小，从而 R^2 必然大。不过，这两个量从两个角度给出我们定量的概念。 R^2 的大小给出了总体上拟合程度的好坏， s 给出了观测点与回归曲线偏离的一个量值。所以，通常在实际问题中两者都求出，供使者从不同角度去认识所拟合的曲线回归。

3) F 检验（类似于一元线性回归中的 F 检验），其数学表达式为：

$$F = \frac{SSR / 1}{SSE / (n-2)}$$

$$\text{其中, } SST = \sum_{i=1}^n (y_i - \bar{y})^2, SSE = \sum_{i=1}^n (y_i - \hat{y}_i)^2, SSR = SST - SSE.$$

对于一元线性回归，通常有三个主要任务：

1) 利用样本观测值对回归系数 β_0 、 β_1 和 σ 做点估计，由于我们所计算出的 $\hat{\beta}_0$ 、 $\hat{\beta}_1$ 仍然是随机变量，因此要对 $\hat{\beta}_0$ 、 $\hat{\beta}_1$ 取值的区间进行估计，如果区间估计值是一个较短的区间表模型精度较高。

2) 对方程的线性关系做显著性检验，反映模型是否具有良好线性关系可通过相关系数的值及 F 值观察。

3) 当可以确定 Y 与 x 之间的函数后，就可以利用该模型对 Y 进行预测。

现在通过实例来说明如何进行一元线性回归：

【例 7-1】近 10 年来，某市社会商品零售总额与职工工资总额（单位：亿元）的数据表 7-1，请建立社会商品零售总额与职工工资总额数据的回归模型。

表 7-1 商品零售总额与职工工资总额

职工工资总额	23.8	27.6	31.6	32.4	33.7	34.9	43.2	52.8	63.8	73.4
商品零售总额	41.4	51.8	61.7	67.9	68.7	77.5	95.9	137.4	155.0	175.0

该问题是典型的一元回归问题，但先要确定是否是线性的，当确定是线性后就可以利用上面的方法建立它们之间的回归模型，具体实现的 MATLAB 代码和各部分代码的执行结果如下：

(1) 输入数据

```
clc, clear all, close all
x=[23.80,27.60,31.60,32.40,33.70,34.90,43.20,52.80,63.80,73.40];
y=[41.4,51.8,61.70,67.90,68.70,77.50,95.90,137.40,155.0,175.0];
```

一元线性回归数学模型：



$$Y = \beta_0 + \beta_1 x + \varepsilon$$



关键注意点：借助已有数据获得其中的参数值。



一元线性回归的作用：

- 1、更好的解释数据背后的现象；
- 2、预测。

2、整理多元线性回归(J p136)的基本数学模型;

7.2 多元回归

7.2.1 多元线性回归

设 Y 是一个可观测的随机变量, 它受到 $p(p > 0)$ 个非随机变量因素 $X_1, X_2, \dots, X_p$ 和随机误差 ε 的影响。若 Y 与 $X_1, X_2, \dots, X_p$, 有如下线性关系:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_p X_p + \varepsilon$$

其中, $\beta_0, \beta_1, \beta_2, \dots, \beta_p$ 是固定的未知参数, 称为回归系数; ε 是均值为 0、方差为 $\sigma^2 (\sigma > 0)$ 的随机变量; Y 称为被解释变量; $X_1, X_2, \dots, X_p$ 称为解释变量。此模型称为多元线性回归模型。

自变量 $X_1, X_2, \dots, X_p$ 是非随机的且可精确观测, 随机误差 ε 代表其随机因素对因变量 Y 产生的影响。

对于总体 $(X_1, X_2, \dots, X_p; Y)$ 的 n 组观测值 $(x_{i1}, x_{i2}, \dots, x_{ip}; y_i) (i = 1, 2, \dots, n; n > p)$, 应满足式:

$$\begin{cases} y_1 = \beta_0 + \beta_1 x_{11} + \beta_2 x_{12} + \dots + \beta_p x_{1p} + \varepsilon_1 \\ y_2 = \beta_0 + \beta_1 x_{21} + \beta_2 x_{22} + \dots + \beta_p x_{2p} + \varepsilon_2 \\ \dots \\ y_n = \beta_0 + \beta_1 x_{n1} + \beta_2 x_{n2} + \dots + \beta_p x_{np} + \varepsilon_n \end{cases}$$

其中 $\varepsilon_1, \varepsilon_2, \dots, \varepsilon_n$, 相互独立, 且设 $\varepsilon_i \sim N(0, \sigma^2) (i = 1, 2, \dots, n)$, 记

$$Y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}, X = \begin{bmatrix} 1 & x_{11} & x_{12} & \dots & x_{1p} \\ 1 & x_{21} & x_{22} & \dots & x_{2p} \\ \vdots & \vdots & \vdots & \dots & \vdots \\ 1 & x_{n1} & x_{n2} & \dots & x_{np} \end{bmatrix}, \beta = \begin{bmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_p \end{bmatrix}, \varepsilon = \begin{bmatrix} \varepsilon_1 \\ \varepsilon_2 \\ \vdots \\ \varepsilon_n \end{bmatrix}$$

则模型可用矩阵形式表示为:

$$Y = X\beta + \varepsilon$$

其中, Y 称为观测向量; X 称为设计矩阵; β 称为待估计向量; ε 是不可观测的 n 维随机向量, 它的分量相互独立, 假定 $\varepsilon \sim N(0, \sigma^2 I_n)$ 。

建立多元线性回归建模的基本步骤如下:

- 1) 对问题进行分析, 选择因变量与解释变量, 作出因变量与各解释变量的散点图, 初步设定多元线性回归模型的参数个数。
- 2) 输入因变量与自变量的观测数据 (y, X) , 计算参数的估计。
- 3) 分析数据的异常点情况。
- 4) 作显著性检验, 若通过, 则对模型作预测。
- 5) 对模型进一步研究, 如残差的正态性检验、残差的异方差检验、残差的自相关性检验等。

对于多元线性回归, 依然可以使用前面介绍的 regress 函数来执行, 现在举例说明如何应

多元线性回归数学模型:



$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_p X_p + \varepsilon$$



关键点: $\beta_1 - \beta_i$ 的含义是每个解释变量的权重。



线性的加减仍是线性, 故一元线性回归模型评估方法仍然适用于多元线性回归模型。

3、整理Logistic回归的基本数学模型（J p144），充分理解Logistic回归方法的含义（J p145）；

7.4 Logistic 回归

7.4.1 Logistic 模型

在回归分析中，因变量 y 可能有两种情形：① y 是一个定量的变量，这时就用通常的 regress 函数对 y 进行回归；② y 是一个定性的变量，比如 $y = 0$ 或 1 ，这时就不能用通常的 regress 函数对 y 进行回归，而是使用所谓的 Logistic 回归。Logistic 方法主要应用在研究某些现象发生的概率 p ，比如股票涨还是跌，公司成功或失败的概率。除此之外，本章还讨论概率 p 与哪些因素有关。Logistic 回归模型的基本形式为：

$$p(Y=1 | x_1, x_2, \dots, x_k) = \frac{\exp(\beta_0 + \beta_1 x_1 + \dots + \beta_k x_k)}{1 + \exp(\beta_0 + \beta_1 x_1 + \dots + \beta_k x_k)}$$

分类器 (数据拟合)

其中， $\beta_0, \beta_1, \dots, \beta_k$ 为类似于多元线性回归模型中的回归系数。

$$\begin{cases} P(Y=1 | x_1, x_2, \dots) = \frac{e^x}{1+e^x} \\ x = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_k x_k \end{cases}$$

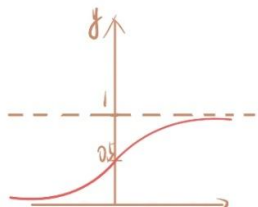
Logistic回归的特殊性：

在回归分析中，因变量 y 可能有两种情形：

1. 当 y 是一个定量的变量（连续的），这时就用通常的 regress 函数对 y 进行回归；
2. 当 y 是一个定性的变量，比如 $y=0$ 或 1 （离散的），这时就不能用一般的 regress 函数对 y 进行回归，而是用 Logistic 回归。例如，预测股票的涨和跌（二分类问题）、是否给客户贷款等。

3、整理Logistic回归的基本数学模型 (J p144)，充分理解Logistic回归方法的含义(J p145);

$$p = \frac{e^x}{1+e^x}$$
$$p(1+e^x) = e^x$$
$$p + pe^x = e^x$$



第7章 数据回归方法 145

该式表示当变量为 $x_1, x_2, \dots, x_k$ 时，自变量 p 为 1 的概率。对该式进行对数变换，可得：

$$e^x = \frac{p}{1-p}$$
$$x = \ln \frac{p}{1-p}$$

$$\ln \frac{p}{1-p} = \beta_0 + \beta_1 x_1 + \dots + \beta_k x_k$$

至此，我们会发现，只要对因变量 p 按照 $\ln(p/(1-p))$ 的形式进行对数变换，就可以将 Logistic 回归问题转化为线性回归问题，此时就可以按照多元线性回归的方法很容易地得到回归参数。但很快又会发现，对于定性实践， p 的取值只有 0,1，这就导致 $\ln(p/(1-p))$ 形式失去意义。为此，在实际应用 Logistic 模型的过程中，常常不是直接对 p 进行回归，而是先定义一种单调连续的概率函数 π ，令

$$\pi = p(Y=1 | x_1, x_2, \dots, x_k), \quad 0 < \pi < 1$$

有了这样的定义，Logistic 模型就可变形为：

$$\ln \frac{\pi}{1-\pi} = \beta_0 + \beta_1 x_1 + \dots + \beta_k x_k, \quad 0 < \pi < 1$$

虽然形式相同，但此时的 π 为连续函数。然后只需要对原始数据进行合理的映射处理，就可以用线性回归方法得到回归系数。最后再由 π 和 p 的映射关系进行反射而得到 p 的值。下面以一个实例来更具体地介绍如何用 MATLAB 进行 Logistic 回归分析。

$$P(Y=1 | x_1, x_2, \dots, x_k) = \frac{\exp(\beta_0 + \beta_1 x_1 + \dots + \beta_k x_k)}{1 + \exp(\beta_0 + \beta_1 x_1 + \dots + \beta_k x_k)}$$

该式表示当变量为 $x_1, x_2, \dots, x_k$ 时，自变量 p 为 1 的概率。对该式进行对数变换，可得：

$$\ln \frac{p}{1-p} = \beta_0 + \beta_1 x_1 + \dots + \beta_k x_k$$

令 $\pi = P(Y=1 | x_1, x_2, \dots, x_k)$, $0 < \pi < 1$ ，Logistic 模型就可变形为：

$$\ln \frac{\pi}{1-\pi} = \beta_0 + \beta_1 x_1 + \dots + \beta_k x_k$$

然后只需要对原始数据进行合理的映射处理，就可以用线性回归方法得到回归系数。最后再由 π 和 p 的映射关系进行反射而得到 p 的值。

3、整理Logistic回归与普通一元线性回归、多元线性回归的联系与区别； 概念阐述

	一元线性回归	多元线性回归	Logistic回归
数学模型	$Y = \beta_0 + \beta_1 x + \varepsilon$	$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p + \varepsilon$	$P(Y = 1 \mid x_1, x_2, \cdots, x_k) = \frac{\exp(\beta_0 + \beta_1 x_1 + \cdots + \beta_k x_k)}{1 + \exp(\beta_0 + \beta_1 x_1 + \cdots + \beta_k x_k)}$
核心内容	只有 1 个自变量，拟合一条直线，预测连续型因变量。	多个自变量，拟合一个平面，预测连续型因变量。	本质是线性模型套了一层 连接函数，用于预测二分类。
作用	预测数值	更精细的数值预测（如用年龄、学历、工作年限预测工资）。	判断是/否（患病与否、违约与否、点击与否）。

3、整理Logistic回归与普通一元线性回归、多元线性回归的联系与区别；

联系与区别

		一元线性回归	多元线性回归	Logistic回归
联系		1、都属于广义线性模型家族 2、都用线性组合 $\beta_0 + \beta_1 x_1 + \dots$ 作为核心 3、都可做显著性检验、系数解释、拟合优度分析		
区别	因变量类型不同	连续数值型	连续数值型	二分类
	结果表示不同	预测结果值	预测结果值	预测结果概率
	假设分布求解	最小二乘法	最小二乘法	极大似然估计法
示例		用身高预测体重	用身高、年龄、性别预测体重	用血压、血糖、年龄预测是否患糖尿病（0 = 不患，1 = 患）

4、阅读《白话机器学习》，第六章“回归”，并完成读书笔记。

第6章 回归分析

趋势线

一般的趋势往往只涉及单个预测变量，这个变量用来产生预测结果，如使用时间来预测一家公司的股票价格。

回归分析不但可以通过考虑更多预测变量改善预测结果，而且还可以比较各个预测变量的强度。

梯度下降法

具体步骤

先初步猜测合适的权重组合

在通过一个迭代过程，把这些权重应用于每个数据点做预测；

然后调整权重，以减少整体预测误差。

局限性

在更复杂的模型中，梯度下降法的结果可能会受到“下山起点”（初始参数值）的影响。

优化方法

随机梯度下降法：每次迭代并不是采用所有数据点，而是只从中选取一个来调整参数。这样就引入了多变性，有助于算法逃离凹坑。

回归系数

基本定义

表示某个预测变量相比于其他预测变量的影响大小；表示相关预测变量的增加值，而非绝对预测强度。

标准化回归系数

统一预测变量的度量单位。

相关系数

基本定义

当只存在一个预测变量时，该预测变量的标准化回归系数也被称作相关系数。

关联方向

相关系数为正表示预测变量和结果变化的方向一致；

相关系数为负表示预测变量和结果变化的方向相反；

关联强度

r 的绝对值越接近1，预测变量起的作用就越大；

若 r 值为0则表示预测变量和结果之间不存在关系；

回归分析的局限性

对异常值敏感

需要在做进一步分析之前，先用散点图找出异常值

造成相关预测变量的权重失真

在分析之前把相关预测变量排除，或者使用更高级的技术，如套索回归或岭回归；

弯曲的趋势

并不说明存在因果关系

阅读总结

本章内容通过预测房价案例向我们由浅及深介绍了回归分析。首先分别通过单一预测变量的趋势线回归指出房价影响的两个因素平均房间数和周围居民收入；这部分内容即详细形象的阐述了一元线性回归的内容；

而后教材将两个因素根据不同权重用作组合变量预测房价，并发现该组合变量的预测准确度更高（误差更小）这部分内容也就是形象具体地介绍了多元线性回归方法；

紧接着教材又向我们介绍确定组合变量权重的方法是梯度下降法，而针对复杂模型梯度下降法可能会受到初始参数值的影响，因而更适合随机梯度下降法来确定权重。

此外，教材还向我们介绍了回归系数和相关系数的概念，相关系数也就是预测变量的标准化回归系数。

最后，教材向我们指出了回归分析虽具有优势但也存在一定的局限性，形如对异常值敏感，容易造成相关预测变量权重失真，这也就警示我们在进行回归分析之前需要对数据进行谨慎的预处理以达到最佳回归分析预测效果。

4、阅读《白话机器学习》，第六章“回归”，并完成读书笔记。

第6章 回归分析

6.1 趋势线

趋势线是做预测时常用的工具，它们很容易生成，也很容易理解。只要翻阅每天的报纸，就会看到大量趋势线图表，涉及的主题各种各样，从股票价格到天气预报。

一般的趋势往往只涉及单个预测变量，这个变量用来产生预测结果，比如使用时间（预测变量）预测一家公司的股票价格（预测结果）。不过，通过添加更多预测变量，可以改善预测结果，比如除了时间之外，还使用销售收入来一起预测股票价格。

回归分析不但可以通过考虑更多预测变量改善预测结果，而且还可以比较各个预测变量的强弱。

为了理解回归分析的原理，让我们看一个预测房价的例子。

6.2 示例：预测房价

本示例使用的是20世纪70年代美国波士顿房价的相关数据及预测变量。经过初步分析发现，对房价影响最大的两个因素是房间数以及周围低收入居民所占的比例。

从图6-1可以看出，价格较高的房子通常房间数较多。为了预测房价，我们画了一条趋势线（图中的蓝线）。这条趋势线亦称最佳拟合线，大部分数据点都分布在这条线附近。由此预测，一所拥有8个房间的房子售价大约是38150美元。

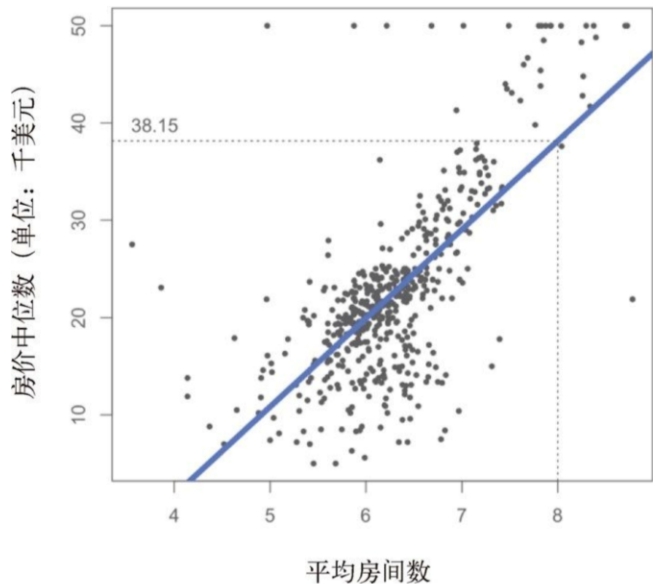


图6-1 房价和房间数的关系

除了房间数之外，房价还受周围居民收入的影响。对于一所房子，其周围低收入居民占的比例越大，房价就越低，如图6-2所示。图6-2a中的趋势线稍有弯曲，通过针对预测变量值应用对数变换，数据点和趋势线能更好地保持一致，如图6-2b所示。

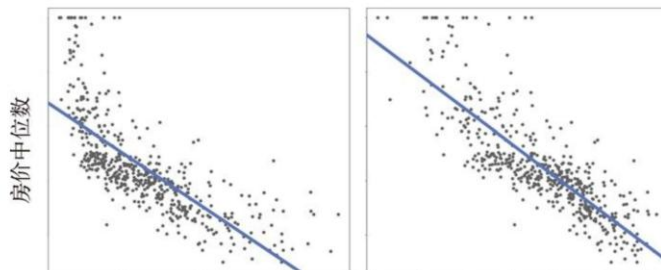


图6-2 房价和低收入居民占比的关系

经过观察可以发现，相比于图6-1，图6-2b中有更多数据点集中在趋势线附近，这说明周围居民的富裕程度对房价的影响要比房间数的影响大。为了提高房价预测结果的准确度，可以把房间数和周围居民的富裕程度结合起来并作为其作为一个预测变量使用。不过，由于后者对房价的影响要比前者大，因此把两者简单地加起来并不合理。合理的做法是给通过周围居民富裕程度所做的预测赋予更高的权重，如图6-3所示。

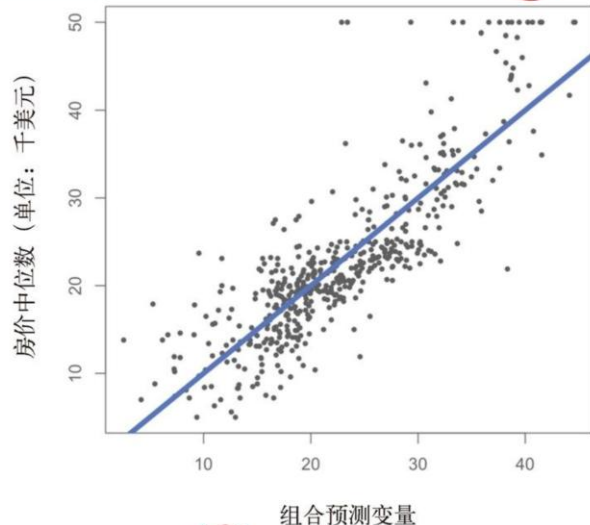


图6-3 房价和带权重的组合预测变量的关系，组合预测变量由房间数和周围居民的富裕程度组合而成

图6-3反映的是房价和带有最佳权重的组合预测变量的关系。该组合预测变量由房间数和周围居民的富裕程度这两个预测变量组合而成。请注意，相比之前，图中的数据点离趋势线更近，因此通过这条趋势线所做的预测可能是最准确的。为了验证这一点，可以比较使用3条趋势线所得到的平均预测误差，如表6-1所示。

表6-1 使用3条趋势线所得到的平均预测误差

	预测误差 (单位：千美元)
房间数	4.4
周围居民的富裕程度	3.9
房间数和周围居民的富裕程度	3.7 组合变量误差最小

显而易见，通过带权重的组合变量能够得到更准确的预测结果。但是，我们在使用过程中要回答如下两个问题。

- (1) 如何得到最优权重组合？
- (2) 如何解释它们？

6.3 梯度下降法

在回归分析中，预测变量的权重是主要参数，通过解方程就可以直接求得最优权重。不过，由于回归分析简单并且适合用于阐释概念，因此我们将用它来解释另外一个优化参数的方法。这个方法就是梯度下降法，一般在无法直接得到参数时使用。

简单地说，梯度下降法先初步猜测合适的权重组合，再通过一个迭代过程，把这些权重应用于每个数据点做预测，然后调整权重，以减少整体预测误差。

这个过程类似于一步步走到山底下。每走一步，梯度下降法都要判断从哪个方向下是最陡峭的，然后朝着那个方向重新校准权重。最终，我们会到达最低点，这个点的预测误差最小，图6-4描绘了一条经过优化的回归趋势线如何与梯度上的最低点相对应。

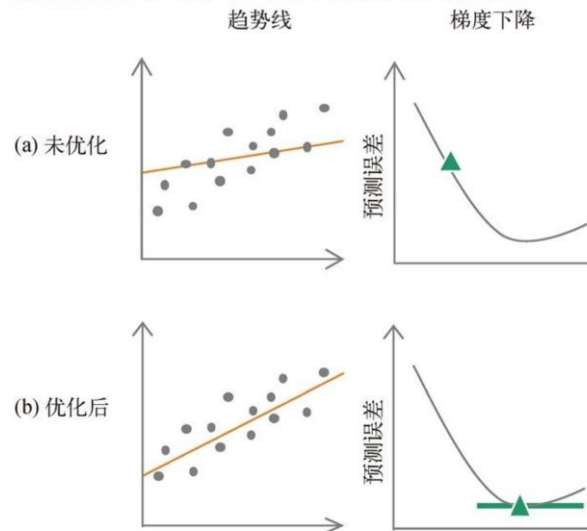


图6-4 趋势线通过梯度下降法达到最优化

除了回归之外，梯度下降法也能用来优化其他模型中的参数，比如第8章讲的支持向量机和第11章讲的神经网络。然而，在这些更为复杂的模型中，梯度下降法的结果可能会受到“下山起点”（即初始参数值）的影响。比如，假设起点下方恰好有一个小凹坑，那么梯度下降法可能会将其误认为是最低点，如图6-5所示。

4、阅读《白话机器学习》，第六章“回归”，并完成读书笔记。

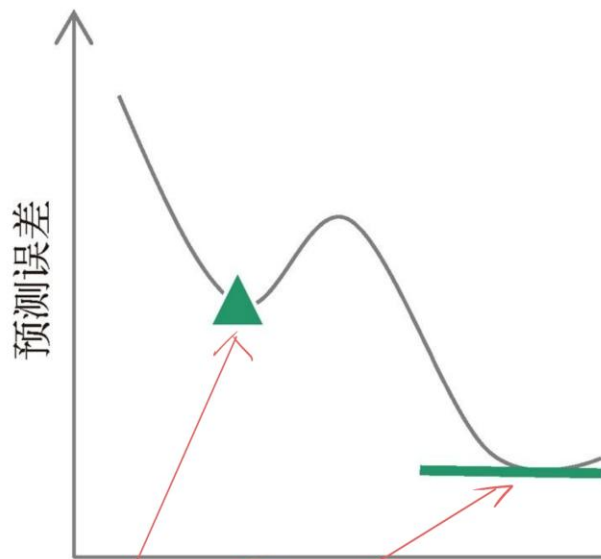


图 6-5 凹坑（绿色三角形）可能会被误认为是最优点，而真实的最优点在更下方（绿色水平线）

为了降低陷入这种凹坑的风险，可以使用另外一种方法——**随机梯度下降法**。在这种方法中，每次迭代并不是采用所有数据点，而是只从其中选取一个来调整参数。这样做就引入了多变性，有助于算法逃离凹坑。虽然从这个随机过程中得到的最终参数值可能不是最优的，但与最优值很接近，精度还是相当不错的。

梯度下降法的这个缺点通常只出现在更为复杂的模型中，做回归分析时根本无须担心这一点。

6.4 回归系数

在为回归预测变量求得最佳权重之后，需要对它们进行解释。

回归预测变量权重的正式名称是**回归系数**，它表示某个预测变量相比于其他预测变量的影响大小。换言之，它表示相关预测变量的增加值，而非绝对预测强度。

举例来说，如果使用房屋的建筑面积和房间数来预测房价，那么房间数的权重也许可以忽略不计。因为房间数在衡量房屋大小方面的作用与建筑面积有重叠，所以它对整个预测能力的贡献很小。

预测变量的度量单位不同也会影响对回归系数的解释。比如，对于同一个预测变量，以米为度量单位时的权重是以厘米为度量单位时的 100 倍。为了避免这个问题，应该在回归分析之前先对预测变量的度量单位进行标准化。标准化类似于统一使用百分位数来表示每个变量。经过标准化之后，预测变量的系数被称为**标准化回归系数**，可以用来做更准确的比较。

在预测房价的例子中，两个预测变量（房间数和周围低收入居民所占比例）都经过了标准化，权重比为 2.7:6.3。这意味着在预测房价时，周围低收入居民所占比例比房间数起更大的作用。回归方程如下所示。

$$\text{房价} = 2.7(\text{房间数}) - 6.3(\text{低收入居民所占比例})$$

请注意，在这个方程中，低收入居民所占比例的权重前面有一个负号，这表示权重为负。这是因为该预测变量和房价是**负相关关系**，这一点可以从图 6-2 中向下倾斜的趋势线看出来。

6.5 相关系数

当只存在一个预测变量时，该预测变量的标准化回归系数也被称为**相关系数**，记作 r ，如图 6-6 所示。相关系数的取值范围为 -1 到 1，它提供了两部分信息。

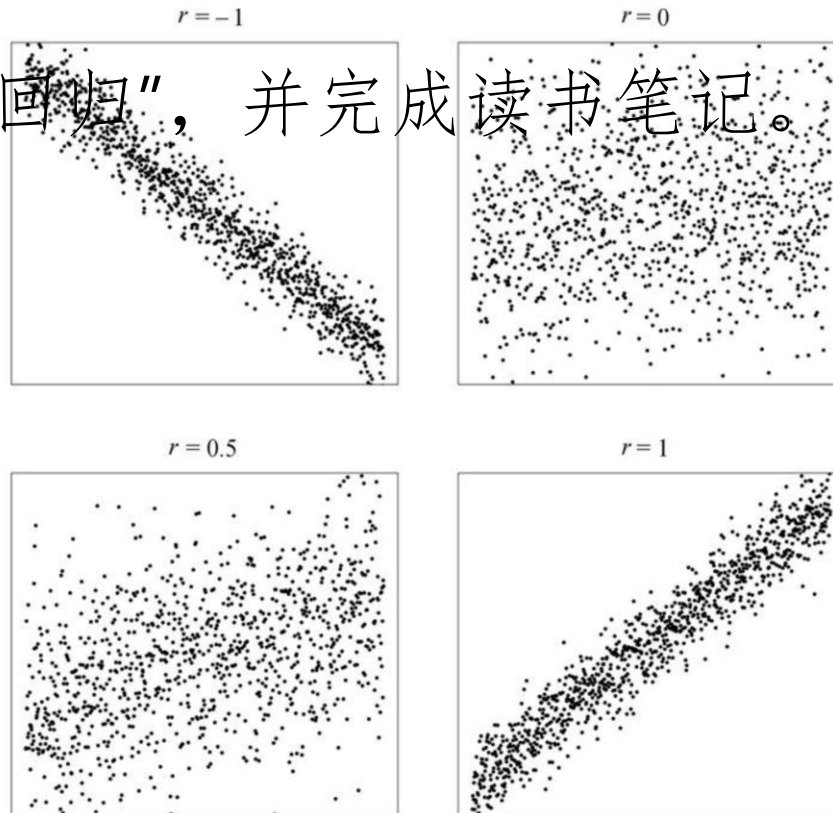


图 6-6 不同的相关系数下数据的分布情况

- **关联方向**：相关系数为正表示预测变量和结果变化的方向一致；为负则表示两者变化方向相反。房价和房间数是正相关关系，和周围低收入居民所占比例是负相关关系。
- **关联强度**： r 值越接近 -1 或 1，预测变量起的作用就越大。例如，图 6-1 中趋势线表示的相关系数是 0.7，而图 6-2b 中的是 -0.8。这意味着在预测房价时，相比于房间数，周围低收入居民所占比例起着更大的作用。若 r 值为 0，则表示预测变量和结果之间不存在关系。因为相关系数表示单个预测变量的**绝对强度**，所以相比于回归系数，相关系数在对预测变量进行排序时更可靠。

6.6 局限性

虽然回归分析能够提供丰富的信息，并且计算速度快，但是它本身存在着一定的局限性。

- **对异常值敏感**：由于回归分析平等地对待所有的数据点，因此只要存在几个有异常值的数据点，就会给趋势线造成很大的影响。为了避免出现这种情况，在做进一步分析之前，可以先使用散点图找出异常值。
- **造成相关预测变量权重失真**：如果回归模型包含高度相关的预测变量，那么这些变量的权重会失真，这就是所谓的**多重共线性**问题。为了解决这个问题，可以在分析之前先把相关预测变量排除，或者使用更高级的技术，比如**套索回归**或**岭回归**。
- **弯曲的趋势**：在本章所举的例子中，趋势由直线表示。但是有些趋势可能是弯曲的。对于这种情况，可能需要对预测变量的值进行转换，或者使用支持向量机（详见第 8 章）等其他算法。
- **并不说明存在因果关系**：假设我们发现宠物狗和房价是正相关关系。我们知道养宠物狗不会让房子增值，但是那些养得起狗的家庭往往会有较高的收入，并且很有可能住在房价较高的社区。

尽管回归分析有上述局限性，但是它仍然是做预测时最常用、最易用、最直观的一种技术。仔细理解分析结果，有助于确保结论的准确性。

6.7 小结

- 回归分析用于寻找**最佳拟合线**，使得尽可能多的数据点位于这条线附近（或这条线上）。
- 趋势线由带权重的组合预测变量得到。这些权重被称为回归系数，表示某个预测变量相对于其他预测变量的影响强度。
- 在下面几种情况下，回归分析的效果最好：

(1) 预测变量之间的关系不强；(2) 无异常值；(3) 趋势可以用直线表示。

分类算法 课后作业3

 姓名：张苏芯

 班级：信管2301

 学号：202321054007

整理分类算法的基本逻辑

分类的定义（目的）



对现有的数据进行学习，得到一个目标函数或模型（**分类器**），该模型能把未知类别的样本映射到给定的类别中。

分类的描述



输入数据，或称训练集（**Training Set**），是由一条条数据库记录组成的。每一条记录包含若干个属性（**Attribute**），组成一个特征向量。训练集的每条记录还有一个特定的类标签（**Class Label**）与之对应。

分类器的作用



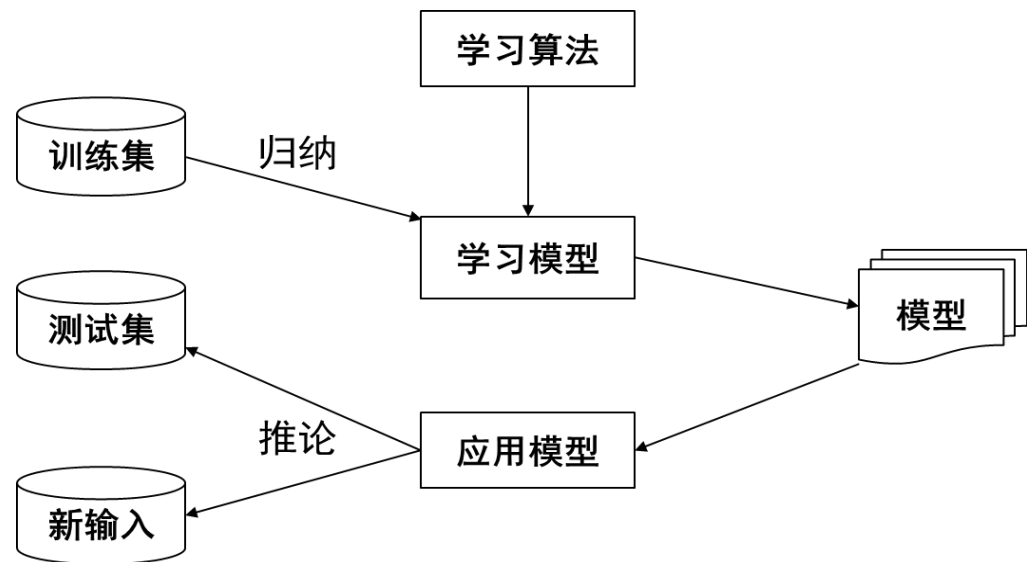
目标函数或规则也称分类模型，其具有两个主要作用，一是描述性建模，即作为解释性的工具，用于区分不同类中的对象；二是预测性建模，即用于预测性未知记录的类标号。

★ 分类是一种预测，不是肯定的，因此不能期待准确率达到100%，但是总是比较选出准确率最高的模型。

整理分类算法的基本逻辑

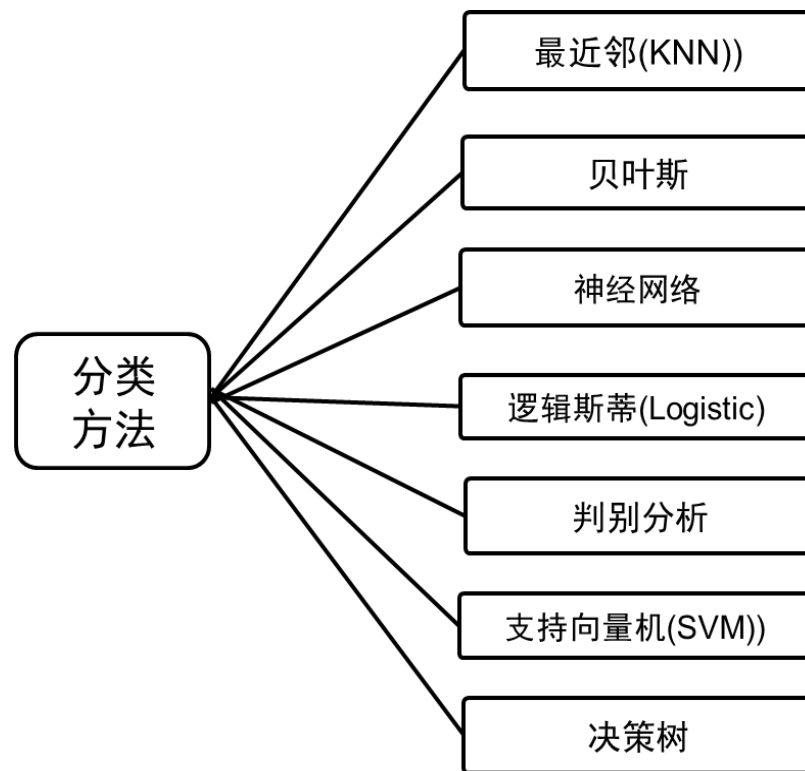
分类的原理

分类方法是一种根据数据集建立分类模型的系统方法，哲学方法都是使用一种学习算法确定分类模型，使其能够很好地拟合输入数据中类标号和属性集之间的联系。



分类的常用方法

- 1、KNN是最简单的算法，基于经验数据的。
- 2、决策树是最具有可解释性的算法（随机森林是它的升级版）
- 3、神经网络（深度学习）是目前最主流的分类算法
- 4、贝叶斯、SVM是曾经的大佬
- 5、Logistic是最经久不衰的，在其他社会科学研究中最常用到



整理分类算法的基本逻辑

分类的评判（混淆矩阵）

混淆矩阵

		预测结果	
		类NO	类YES
实际类别	类NO	TN	FP
	类YES	FN	TP

TP: True Positive; TN: True Negative;
FP: False Positive; FN: False Negative;

8.9 分类的评判

8.9.1 正确率

在介绍系列指标之前，先明确以下4个基本的定义：

1) True Positive (TP)：指模型预测为正(1)的，并且实际上也是正(1)的观察对象的数量。

2) True Negative (TN)：指模型预测为负(0)的，并且实际上也是负(0)的观察对象的数量。

178 ❖ 第二篇 技术篇

指标的关注意点：垃圾邮件，金融欺诈，需减少误报FP，增加P；癌症筛查，需减少漏报FN，增加R；

3) False Positive (FP)：指模型预测为正(1)的，但是实际上是负(0)的观察对象的数量。

4) False Negative (FN)：指模型预测为负(0)的，但是实际上是正(1)的观察对象的数量。

上述4个基本定义可以用一个表格形式简单地体现，如表8-2所示。

基于上面的4个基本定义，可以延伸出下列评价指标：

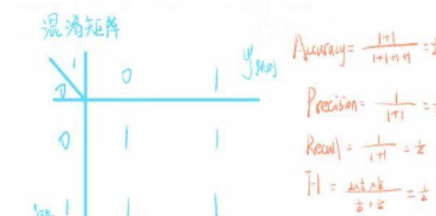


表 8-2 二类问题的混淆矩阵

实际的类 \ 预测的类	类 1	类 0
类 1	TP	FN
类 0	FP	TN

$$Accuracy = \frac{TP+TN}{TP+TN+FP+FN}$$

$$Precision = \frac{TP}{TP+FP}$$

$$Recall = \frac{TP}{TP+FN}$$

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

整理分类算法的几个基本概念

分类器

能够解释原始数据，能够预测新数据的模型。

训练集

用来训练模型的原始数据（划分出一部分）。

类标签

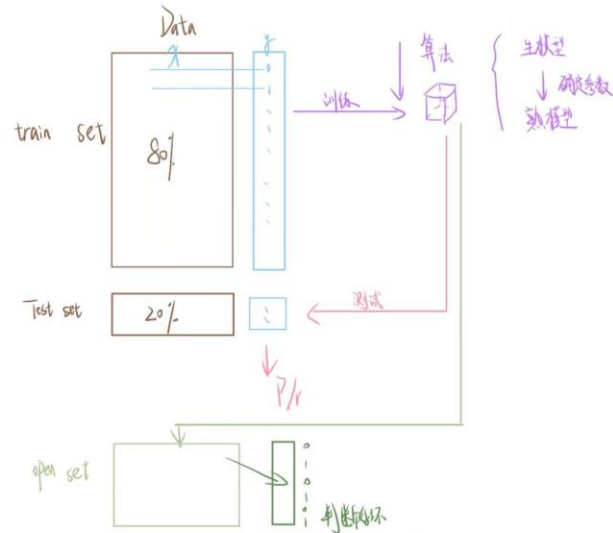
原始数据中已经给每个记录（对象）标记的类别。

测试集

用来测试模型的原始数据（划分出的一部分）。

开放集

使用训练好的模型（分类器）进行分类工作的新数据。



大语言模型 = 海量知识库测试 + 知识重组能力(概率) + 人类提问的智慧

例:

表 7-8 企业还款能力评价表

企业编号	X1	X2	X3	类标签(Y)	预测值
1	-62.8	-89.5	1.7	0	0
2	3.3	-3.5	1.1	0	0
3	-120.8	-103.2	2.5	0	0
4	-18.1	-28.8	1.1	0	0
5	-3.8	-50.6	0.9	0	0
6	-61.2	-56.2	1.7	0	0
7	-20.3	-17.4	1	0	0
8	-194.5	-25.8	0.5	0	0
9	20.8	-4.3	1	0	0
10	-106.1	-22.9	1.5	0	0
11	43	16.4	1.3	1	1
12	47	16	1.9	1	1
13	-3.3	4	2.7	1	1
14	35	20.8	1.9	1	1
15	46.7	12.6	0.9	1	1
16	20.8	12.5	2.4	1	1
17	33	23.6	1.5	1	1
18	26.1	10.4	2.1	1	1
19	68.6	13.8	1.6	1	1
20	37.3	33.4	3.5	1	1
21	-49.2	-17.2	0.3	?	0
22	-19.2	-36.7	0.8	?	0
23	40.6	5.8	1.8	?	1
24	34.6	26.4	1.8	?	1
25	19.9	26.7	2.3	?	1

原始数据

训练集
(原始数据中用于训练模型的部分数据)
此处示例11

测试集
(原始数据中用于测试模型的部分数据)
此处示例17

开放集
(用于测试模型的模型)

整理掌握KNN算法的思想与算法步骤;

8.2 K-近邻

8.2.1 K-近邻原理

前提: 可分类 学习 { ① 合适的K ② 数据空间的分布特征 $\beta_0, \beta_1, \beta_2 \rightarrow$ 参数

K-近邻 (K-Nearest Neighbor, KNN) 算法是一种基于实例的分类方法, 最初由 Cover 和 Hart 于 1968 年提出, 是一种非参数的分类技术。

① K-近邻分类方法通过计算每个训练样例到待分类样品的距离, 取和待分类样品距离最近的 K 个训练样例, ② K 个样品中哪个类别的训练样例占多数, ③ 则待分类元组就属于哪个类别。(效果) 使用最近邻确定类别的合理性可用下面的谚语来说明: “如果走像鸭子, 叫像鸭子, 看起来还像鸭子, 那么它很可能就是一只鸭子”, 如图 8-3 所示。最近邻分类器把每个样例看作 d 维空间上的一个数据点, 其中 d 是属性个数。给定一个测试样例, 我们可以计算该测试样例

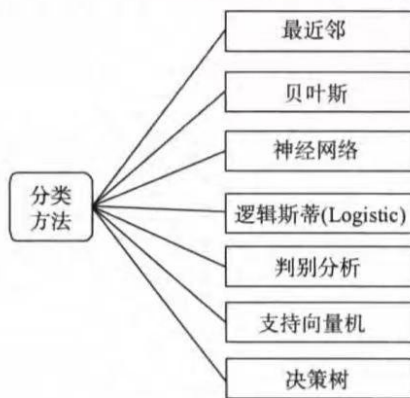


图 8-2 常用的分类方法

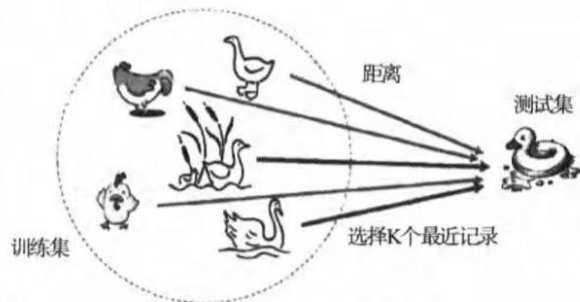


图 8-3 KNN 方法原理示意图

KNN 算法具体步骤如下:

- 步骤 1: 初始化距离为最大值。
- 步骤 2: 计算未知样本和每个训练样本的距离 dist。
- 步骤 3: 得到目前 K 个最近邻样本中的最大距离 maxdist。
- 步骤 4: 如果 dist 小于 maxdist, 则将该训练样本作为 K-最近邻样本。
- 步骤 5: 重复步骤 2~步骤 4, 直到未知样本和所有训练样本的距离都算完。
- 步骤 6: 统计 K 个最近邻样本中每个类别出现的次数。
- 步骤 7: 选择出现频率最大的类别作为未知样本的类别

合适的K值 根据 KNN 算法的原理和步骤可以看出, KNN 算法对 K 值的依赖较高, 所以 K 值的选择非常重要。如果 K 太小, 预测目标容易产生变动性; 相反, 如果 K 太大, 最近邻分类器可能会误分类测试样例, 因为最近邻列表中可能包含远离其近邻的数据点 (见图 8-5)。推定 K 值的有益途径是通过有效参数的数目这个概念, 有效参数的数目是和 K 值相关的, 大致等于 n/K , 其中, n 是这个训练数据集中实例的数目。在实践中往往通过若干次实验来确定 K 值, 取分类误差率最小的 K 值。

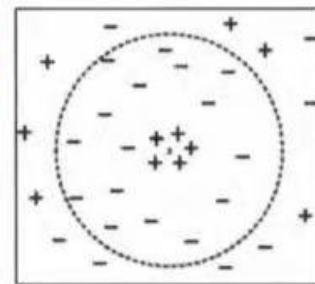
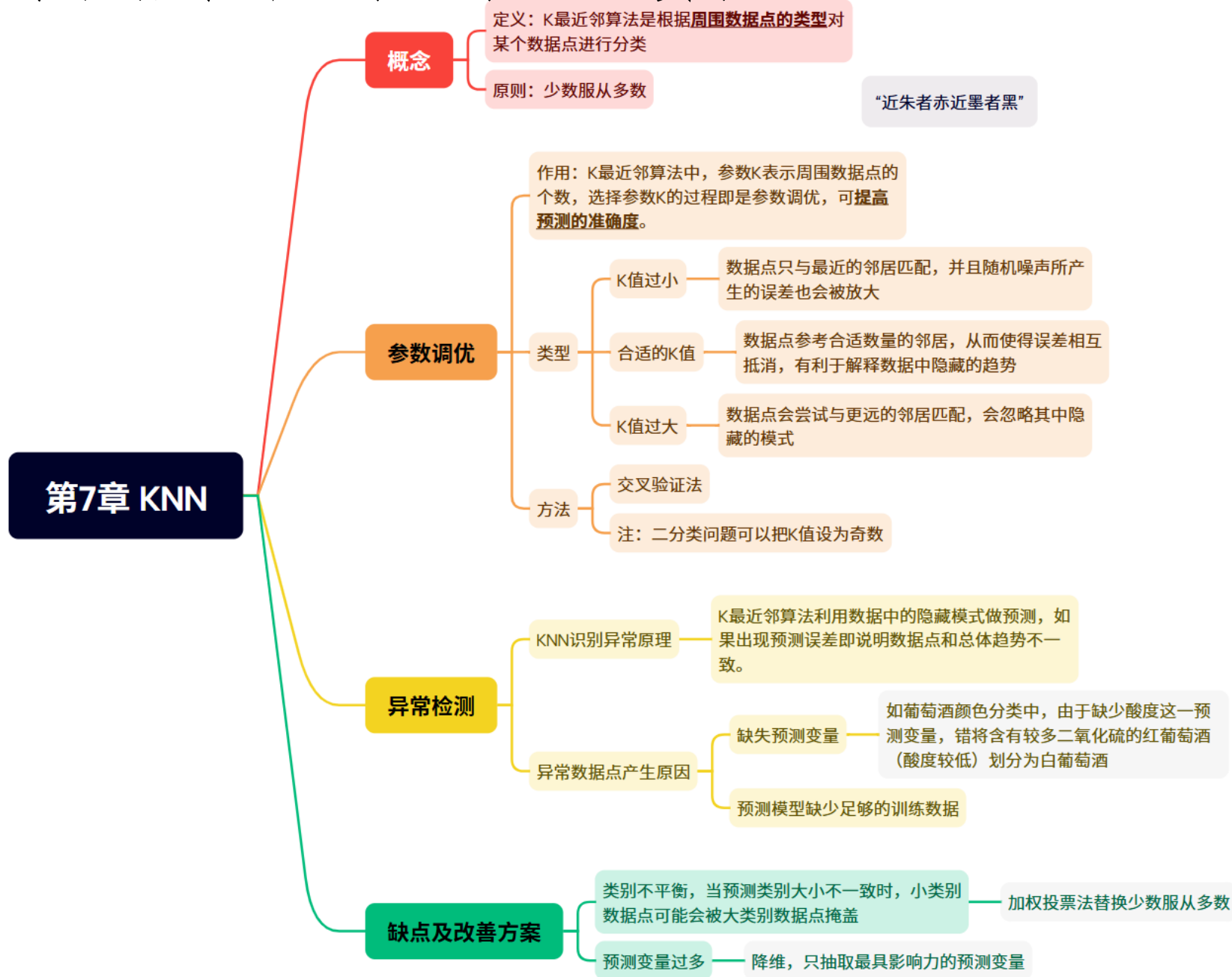


图 8-5 K 较大时的 K-最近邻分类

整理掌握KNN算法的思想与算法步骤；



《白话机器学习》第七章KNN读书笔记。



《白话机器学习》第七章KNN读书笔记。

第7章 k 最近邻算法和异常检测

7.1 食品检测

让我们来聊聊葡萄酒。你是否曾想真正地搞清楚红葡萄酒和白葡萄酒的区别？

有些人想当然地认为，红葡萄酒是用红葡萄酿制的，而白葡萄酒是用白葡萄酿制的。但是这并非完全正确，尽管红葡萄酒不能用白葡萄酿制，可白葡萄酒可以用红葡萄酿制的。

红白葡萄酒最大的区别在于葡萄的发酵方式不同。酿制红葡萄酒时，葡萄汁和葡萄皮是混在一起发酵的，葡萄皮在这个过程中会释放出独特的红色素。酿制白葡萄酒时，则要把葡萄皮去掉，只发酵葡萄汁。

一方面，我们可以根据葡萄酒的颜色推断其酿制过程有无用到葡萄皮；另一方面，葡萄皮会导致葡萄酒的化学成分发生变化，这意味着不用观察葡萄酒的颜色，只通过分析化学成分的含量就能推断出葡萄酒的颜色。

为了检验这个假设，可以使用机器学习中最简单的一种方法： k 最近邻算法。

7.2 物以类聚，人以群分

k 最近邻算法根据周围数据点的类型对某个数据点进行分类。也就是说，如果一个数据点周围有4个红点和1个黑点（如图7-1所示），那么根据少数服从多数的原则，这个数据点很可能就是红色的。

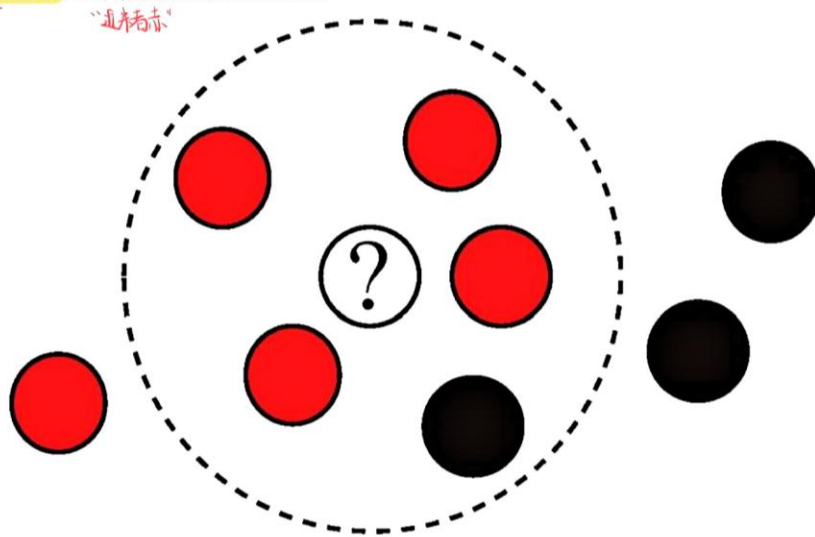


图 7-1 根据周围5个数据点的颜色以及少数服从多数的原则，中心数据点应该被划为红点

在 k 最近邻算法中，参数 k 表示周围数据点的个数。在上面的例子中， k 为5。选择 k 值的过程叫作参数调优，它对预测的准确度起着至关重要的作用。

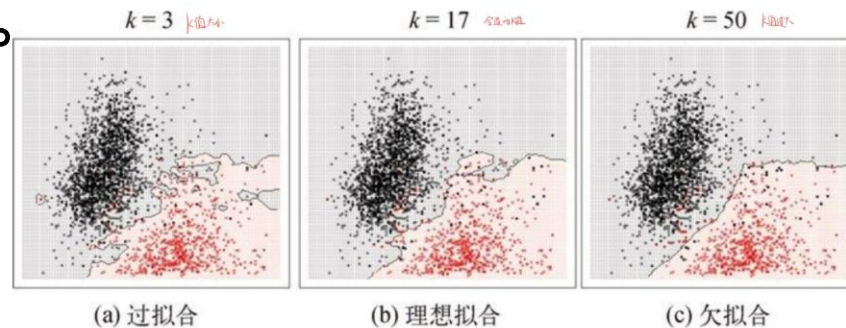


图 7-2 使用不同的 k 值进行拟合。黑色区域中的点被预测为白葡萄酒，红色区域中的点则被预测为红葡萄酒

如果 k 值太小，数据点只与最近的“邻居”匹配，并且随机噪声所产生的误差也会被放大，如图7-2a所示。如果 k 值太大，数据点会尝试与更远的“邻居”匹配，其中隐含的模式会被忽略，如图7-2c所示。

只有当 k 值恰到好处时，数据点才会参考合适数量的“邻居”，这使得误差相互抵消，有利于揭示数据中隐藏的趋势，如图7-2b所示。

为实现理想拟合并把误差降到最低，可以使用交叉验证法对参数 k 进行调优（请参考1.4.3节）。对于二分类问题，可以把 k 设置成一个奇数，以避免出现平局的情况。

除了用来为数据点分类， k 最近邻算法还可以通过统计周围数据点的值来预测连续值。相比于平等看待周围的所有数据点并简单地取平均值，通过使用加权平均值，能够进一步改善预测结果。离数据点越近的“邻居”，其值越能反映该数据点的真实值，因此赋给它的权重应该更大。

7.3 示例：区分红白葡萄酒

回到葡萄酒的例子。通过观察与之有相似化学成分的葡萄酒，可以猜出某款葡萄酒的颜色。

如图7-3所示，我们利用葡萄牙青酒的各种红白变种酒的数据把1599种红葡萄酒和4898种白葡萄酒的化学成分绘制了出来，图中涉及两种化学成分，即氯化物（横轴）和二氧化硫（纵轴）。

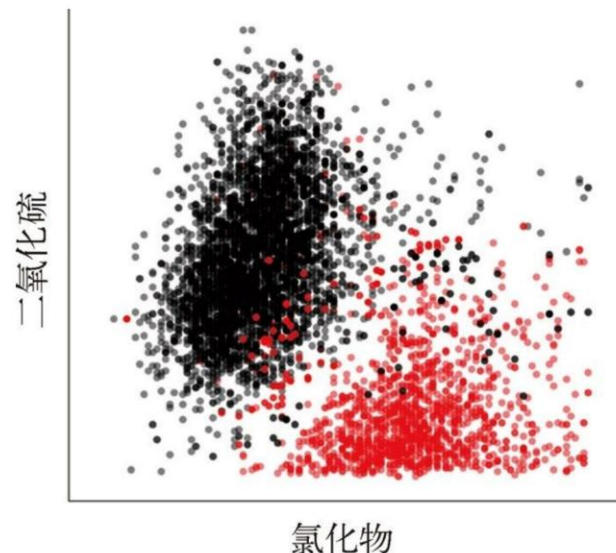


图 7-3 白葡萄酒（黑点）和红葡萄酒（红点）中氯化物和二氧化硫的含量

因为葡萄皮中的矿物质（比如氯化钠，与食盐成分一样）含量较高，所以红葡萄酒中这些成分的含量就相对较高，图中很好地反映出了这一点。葡萄皮还含有天然抗氧化剂，用来使葡萄保持新鲜。白葡萄酒不包含这种成分，所以需要更多的二氧化硫来充当防腐剂。正是这些原因使得红葡萄酒大都集中在图中的右下部分，白葡萄酒则主要集中在左上部分。

《白话机器学习》第七章KNN读书笔记。

在推断含有特定量氯化物和二氧化硫的葡萄酒的颜色时，可以参考与其有相似化学成分含量的葡萄酒的颜色。对图中每个点都这样做一遍，可以画出用以区分红葡萄酒和白葡萄酒的分界线。如图 7-2b 所示，在理想拟合的情况下，推断葡萄酒颜色的准确率超过 98%。

7.4 异常检测

K最近邻算法不仅可以用来预测数据点的类别和取值，还可以用来识别异常，比如检测欺诈行为。而且，在异常检测过程中还可能会有新的发现，比如发现之前被忽略的预测变量。

数据可视化让异常检测变得简单。比如在图 7-3 中，我们能一眼看出哪些酒偏离了它们所属的群组。不过，并非所有数据都可以用二维图表示，尤其是当要检查的预测变量超过两个时，更是如此。这正是 k 最近邻等预测模型大显身手的时候。

因为 K 最近邻算法利用数据中的隐藏模式做预测，所以如果出现预测误差，就说明数据点和总体趋势不一致。事实上，任何能够产生预测模型的算法都可以用来检测异常。比如，在回归分析中，如果某个数据点明显偏离最佳拟合线，那么就会被识别为异常点。

稍微分析一下葡萄酒颜色归类错误时的异常数据，就会发现那些被错划成白葡萄酒的红葡萄酒往往含有较多的二氧化硫。由于这些葡萄酒的酸度较低，因此需要更多的二氧化硫来充当防腐剂。如果知道了这一点，那么我们可能会把葡萄酒的酸度也考虑进去，从而进一步提高预测的准确度。

异常数据点既可能因缺失预测变量所致，也可能因预测模型缺少足够的训练数据所致。我们拥有的数据点越少，就越难发现隐藏于数据中的模式，所以务必确保建模时有足够的样本可用。

一旦找到异常数据点，就要将它们从数据集中移除，然后再训练预测模型。这样做可以减少数据中包含的噪声，进而提高模型的准确度。

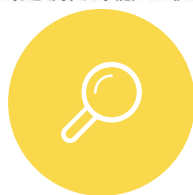
7.5 局限性

尽管 K 最近邻算法简单且实用，但是在如下情形中使用该算法可能无法取得好的效果。

- 类别不平衡：如果待预测的类别有多个，并且在大小方面存在很大的不同，那么那些属于最小类别的数据点可能会被来自更大类别的数据点所掩盖，它们被错误分类的风险更大。为了提高准确度，可以使用加权投票法来取代少数服从多数的原则，这会确保较近数据点类别的权重比较远的更大。
- 预测变量过多：如果待考虑的预测变量太多，在多个维度上识别和处理近邻会导致计算量大增。而且，有些预测变量可能是多余的，它们对提高预测准确度没有用处。为了解决这个问题，可以使用第 3 章介绍的降维技巧，只抽取最具影响力的预测变量用于分析。

7.6 小结

- K 最近邻算法根据周围数据点的类型对某个数据点进行分类。
- K 表示用作参考的数据点的个数，可以使用交叉验证法来确定。
- 当预测变量数目不多，并且类别大小差别不大时，K 最近邻算法才能产生非常好的效果。不准确的分类可能会被标记为潜在异常。



阅读总结

本章通过运用 K 最近邻算法区分白葡萄酒与红葡萄酒的案例详细阐述了 KNN 算法的原理及步骤。即通过两种酒所含化学成分不同，红葡萄酒富含氯化物，白葡萄酒富含二氧化硫这一原理结合 K 最近邻算法进行区分酒的种类。

此外，KNN 除了可以在分类中进行应用，其亦可以在异常检测中发挥其特有优势，并指出异常数据点可能因缺失预测变量和缺少训练数据导致。因此，在进行分类时需考虑上述因素。如，当区分白红葡萄酒缺少“酸度”这一预测变量时，就容易仅凭化学成分单一预测变量将酸度低的红葡萄酒（含较多的二氧化硫）错误地划分为白葡萄酒。

同时本章节最后也揭示出该算法存在的局限性，形如当预测类别过多且大小不一致时，小类别的数据点容易被大类别数据点掩盖从而可能导致错误分类，需采用根据类别大小设置不同权重，并使用加权投票法取代少数服从多数的原则。

《白话机器学习》第九章决策树读书笔记

第9章 决策树

作用

- 预测疾病的存活率
- 估计员工的辞职概率
- 检测器欺诈交易
- 分类问题
- 连续值问题（可转换为分类问题，形如比较高于还是低于平均值）

生成决策树

- 理论基础
 - 相同路径上的数据点彼此是相似的
- 递归拆分
 - 确定一个二元选择题，它能够把数据点拆分为两组，并最大限度地提高每组数据点的同质性
 - 针对每个叶节点重复步骤1，直到满足终止条件。
- 选取终止条件（交叉验证法）
 - 每个叶节点中的数据点全属于同一类或有相同的值
 - 叶节点包含的数据点少于5个
 - 进一步分支会超出阈值并且不能提高同质性

局限性

- 不确定：数据中细微变化可能影响拆分结果，并导致生成决策树截然不同
- 不准确：一开始就使用最佳二元选择题拆分数据点并不能保证结果最准确

决策树多样化

- 随机森林
 - 随机选择不同的二元选择题，生成多棵决策树，综合决策树的预测结果
- 梯度提升
 - 有策略地选择二元选择题，逐步提高预测准确度，将所有预测结果的加权平均数作为最终结果

阅读总结

本章通过运用决策树算法预测泰坦尼克号乘客生还概率的案例详细阐述了决策树的原理及生成决策树的步骤。即通过性别、年龄、舱位等特征，设计了一系列二元选择题，自上而下递归拆分数据，最终来判断生还可能性。

此外，本章节也指出由于递归拆分只用最佳二元选择题来生成决策树，因此不显著的变量不会影响结果，从而使得决策树对异常值有较强的耐扰性这一优点。

但由于决策树容易因数据细微变化产生不稳定结果，且一味追求最佳拆分容易出现过拟合问题。因此，为了弥补这些不足，可采用随机森林、梯度提升等方法进行改善使得决策树多样化，通过多棵决策树组合来提升预测准确度。

《白话机器学习》第九章决策树读书笔记

从决策树可知，对于男孩或者女孩来说，只要不在三等舱，那么活下来的可能性就很大。

决策树有许多用处，比如预测疾病的存活率，估计员工的辞职概率，或者检测欺诈交易等。此外，决策树还可以用来处理分类问题（比如男性和女性）或连续值问题（比如工资）。请注意，连续值问题有时可以转化为分类问题，比如比较高于和低于平均值的值。

标准决策树的每个分支只存在两个答案，比如“是”或“否”。如果有两个以上的答案（比如“是”“否”“有时”），可以沿着分支继续向下添加更多分支（如图 9-3 所示）。

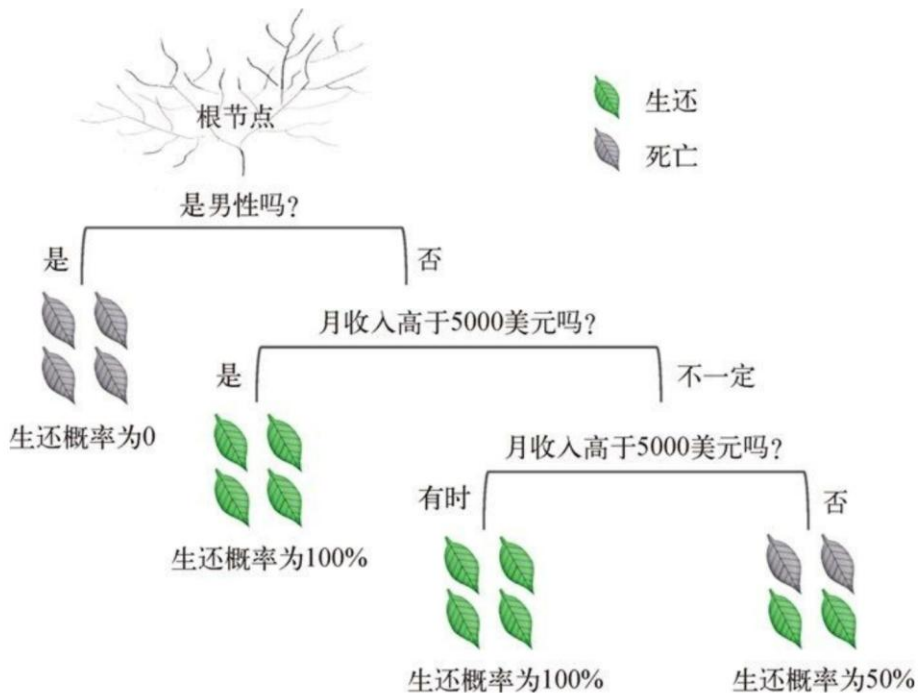


图 9-3 在决策树中测试多个类别

决策树之所以受欢迎，是因为它容易解释。那么，如何生成决策树呢？

9.3 生成决策树

若要生成决策树，首先根据相似性把所有数据点分为两组，然后针对每组重复这个二分过程。每一层叶节点都比上一层包含更少的数据点，但同质性更高。决策树的理论基础是，相同路径上的数据点彼此是相似的。

这个反复拆分数据以得到同质组的过程被称为递归拆分，它只包含如下两个步骤。

步骤 1：确定一个二元选择题，它能够把数据点拆分为两组，并最大限度地提高每组数据点的同质性。

步骤 2：针对每个叶节点重复步骤 1，直到满足终止条件。

图 9-4 展示了一个决策树生成示例。

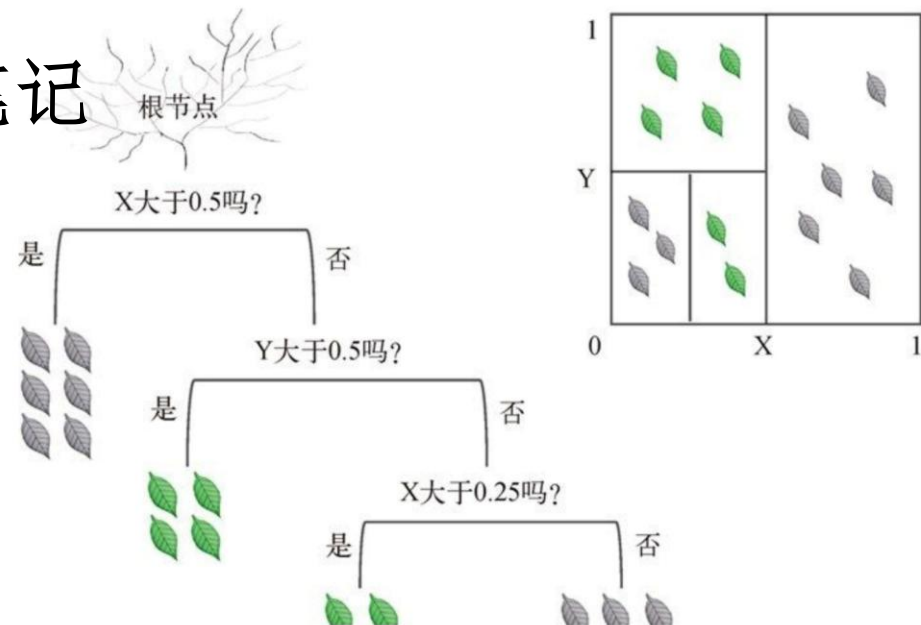


图 9-4 通过决策树拆分数据点并用散点图表示出来

终止条件可能有多，可以使用交叉验证法（详见 1.4.3 节）进行选取。本例有以下终止条件：

- 每个叶节点中的数据点全属于同一类或有相同的值；
- 叶节点包含的数据点少于 5 个；
- 进一步分支会超出阈值并且不能提高同质性。

由于递归拆分只用最佳二元选择题来生成决策树，因此不显著的变量并不会影响结果。而且，二元选择题往往围绕着最重要的值划分数据点，所以决策树对异常值有较强的耐性。

9.4 局限性

虽然决策树容易解释，但存在如下缺点。

- 不稳定：决策树是通过把数据点分组生成的，数据中的细微变化可能影响拆分结果，并导致生成的决策树截然不同。此外，每次拆分数据点时都力求找到最佳拆分方式，这很容易产生过拟合问题（详见 1.3 节）。
- 不准确：一开始就使用最佳二元选择题拆分数据点，并不能保证结果最准确。有时，先用不太有效的分法反而会产生比较好的预测结果。

为了克服上述缺点，每次拆分时可以不采用最佳拆分方式，而是尽量让决策树多样化。然后，综合不同的决策树产生的预测结果，让最终预测结果具有更好的稳定性和准确性。

决策树的多样化方法有如下两种。

- 随机森林：随机选择不同的二元选择题，生成多棵决策树，然后综合这些决策树的预测结果。第 10 章将详解这种方法。
- 梯度提升：有策略地选择二元选择题，以逐步提高决策树的预测准确度；然后将所有预测结果的加权平均数作为最终结果。

虽然随机森林和梯度提升能够产生更准确的预测结果，但是它们往往比较复杂，并且很难进行可视化，因而得名“黑盒”。这也解释了为什么决策树至今仍然是一个广受欢迎的分析工具：它易于可视化，这使我们更容易评估预测变量及其相互作用。

9.5 小结

- 决策树通过询问一系列二元选择题来做预测。
- 若想生成决策树，就要不断拆分数据样本以获得同质组，直到满足终止条件。这个过程被称为递归拆分。
- 虽然决策树易于使用和理解，但是容易造成过拟合问题，导致出现不一致的结果。为了尽量避免出现这种情况，可以采用随机森林等替代方法。

聚类算法 课后作业4

 姓名：张苏芯

 班级：信管2301

 学号：202321054007

1.整理簇的概念 (Jp187-188)

聚类的概念

将物理或抽象对象的集合分成由类似的对象组成的多个类或簇的过程被称为聚类。

对聚类理解的三个层次

层次一

“物以类聚，人以群分”（感官）

层次二

同一个簇中的对象相似度较高，与其它簇中的对象相似度较低。（逻辑）

层次三

同一个分组中的记录越近越好，而不同分组中的记录越远越好。（量化）

在自然科学和社会科学中，存在着大量的聚类问题，其实聚类是一个人们日常生活的常见行为，所谓“物以类聚，人以群分”，其核心思想也是聚类。人们总是不断地改进下意识中的聚类模式来学习如何区分各个事物和人。通过聚类，人们能意识到密集和稀疏的区域，发现全局的分布模式，以及数据属性之间有趣的相互关系。

聚类起源于分类学，在古老的分类学中，人们主要依靠经验和专业知识来实现分类，很少利用数学工具进行定量的分类。随着人类科学技术的发展，对分类的要求越来越高，以致有时仅凭经验和专业知识难以确切地进行分类，于是人们逐渐地把数学工具引用到了分类学中，形成了数值分类学，之后又将多元分析的技术引入到数值分类学形成了聚类。在实践中，聚类往往为分类服务，即先通过聚类来判断事物的合适类别，然后再利用分类技术对新的样本进行分类。

聚类已经广泛地应用在许多应用中，包括模式识别、数据分析、图像处理以及市场研究。作为一个数据挖掘的功能，聚类能作为独立的工具来获得数据分布的情况，观察每个簇的特点，集中对特定的某些簇做进一步的分析，此外，聚类分析还可以作为其他算法的预处理步骤，简化计算量，提高分析效率。本章将介绍聚类的常用方法和典型的应用案例。

层次一：“物以类聚，人以群分”（感官）

9.1 聚类方法概要

层次二：同一个簇中的对象相似度较高，与其它簇中的对象相似度较低（逻辑）

9.1.1 聚类的概念

数据空间

层次三：同一个分组中的记录越近越好，而不同分组中的记录越远越好（量化）

将物理或抽象对象的集合分成由类似的对象组成的多个类或簇（Cluster）的过程被称为聚类（Clustering）。由聚类所生成的簇是一组数据对象的集合，这些对象与同一个簇中的对

（逻辑）

象相似度较高，与其他簇中的对象相似度较低。相似度是根据描述对象的属性值来度量的，距离是经常采用的度量方式。分析事物聚类的过程称为聚类分析又称群分析，它是研究（样品或指标）分类问题的一种统计分析方法。

9.2 K-means 方法

K-均值聚类算法是著名的划分聚类分割方法。划分方法的基本思想是：给定一个有 N 个元组或者记录的数据集，分裂法将构造 K 个分组，每一个分组就代表一个聚类， $K < N$ 。而且这 K 个分组满足下列条件：①每一个分组至少包含一个数据记录；②每一个数据记录属于且仅属于一个分组；对于给定的 K ，算法首先给出一个初始的分组方法，以后通过反复迭代的方法改变分组，使得每一次改进之后的分组方案都较前一次好，而所谓好的标准就是：同一分组中的记录越近越好（已经收敛，反复迭代至组内数据几乎无差异），而不同分组中的记录越远越好。（量化）

1. 整理簇的概念（Jp187-188）

簇的“定义”

簇的定义是不精确的，最好的定义依赖于数据特性和期望的结果。

簇的形象表现

其形象表现在空间分布上不是确定的，而是呈各种不同的形状，在二维平面里就可以有各种不同的形状；在多维空间里有更多的形状。

对应长的不一样的对象，会有不同的算法更合适去应对。

总体趋势

同一个簇的样本在空间上是靠拢在一起的。

在许多应用中，簇的概念都没有严格的定义。为了理解确定簇构造的困难性，可参考图 9-1。该图显示了 18 个点和将它们划分成簇的 3 种不同方法。标记的形状指示簇的隶属关系。图 9-1b 和图 9-1d 分别将数据划分成两部分和六部分。然而，将 2 个较大的簇都划分成 3 个子簇可能是人的视觉系统造成的假象。此外，说这些点形成 4 个簇（如图 9-1c 所示）可能也不无道理。该图表明簇的定义是不精确的，而最好的定义依赖于数据的特性和期望的结果。另外，簇的形象表现在空间分布上也不是确定的，而是成各种不同的形状，在二维平面里就可以有各种不同的形状，如图 9-2 所示，在多维空间里，更是有更多的形状。所以簇的定义，也需要具体情况具体分析，但总的趋势是，同一个簇的样本在空间上是靠拢在一起的。

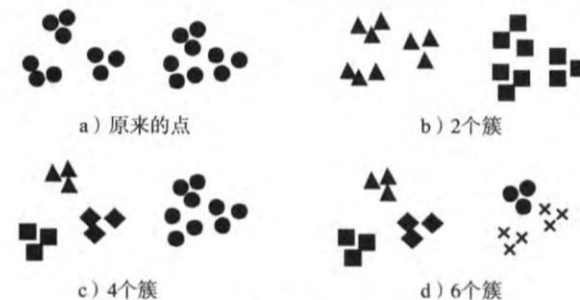


图 9-1 相同点集的不同聚类方法

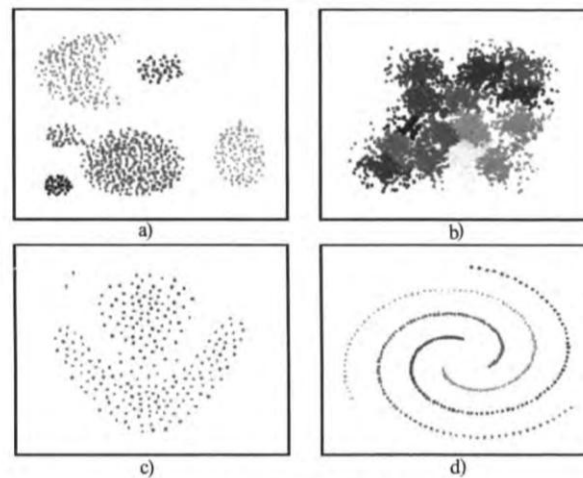
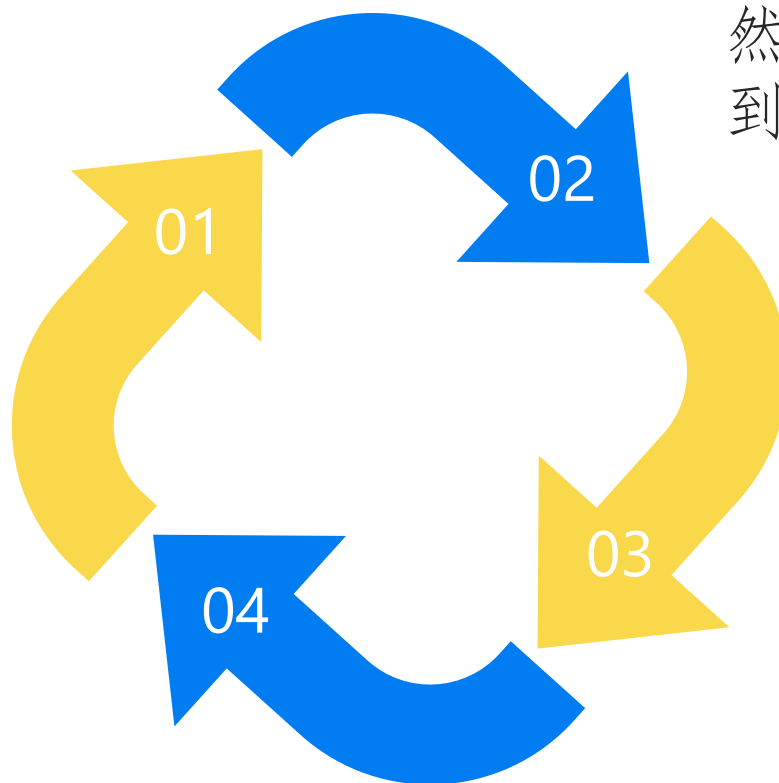


图 9-2 常见的类别特征

3.整理K-means算法的原理和步骤（Jp192）

01
首先随机从数据集中选取
K个点，每个点初始地代
表每个簇的聚类中心



02
然后计算剩余各个样本
到聚类中心的距离，将
它赋给最近的簇

03
接着重新计算每一簇的
平均值，作为新的聚类
中心；

04

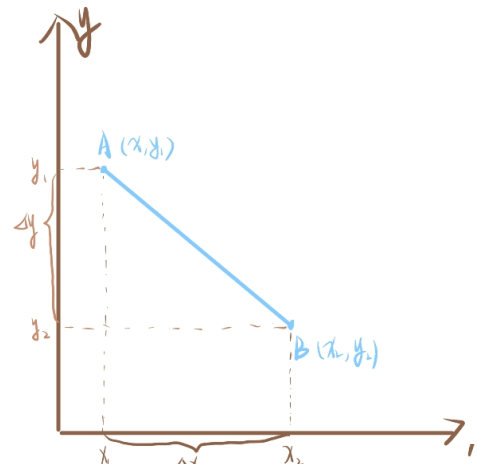
整个过程不断重复，如果相邻两次调整没有明显变化（点的归属或者聚类中心不再改变），说明数据聚类形成的簇已经收敛。

2. 整理类度量的两种基本方法(欧式距离&余弦距离) (Jp189-190)

类的度量方法

欧式距离

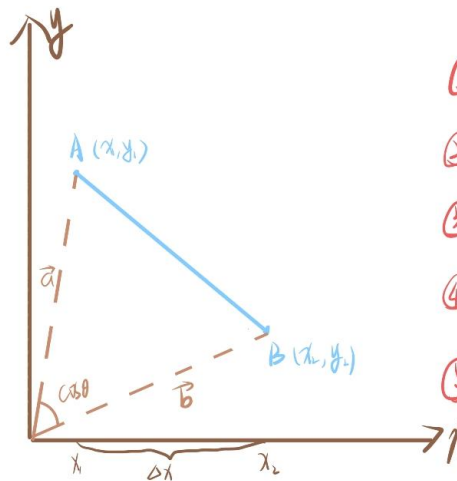
$$d_{ij}(2) = \left[\sum_{k=1}^p |x_{ik} - x_{jk}|^2 \right]^{1/2}, \quad i = 1, 2, \dots, n; j = 1, 2, \dots, n$$



$$d_{AB} = \sqrt{\frac{(x_1 - x_2)^2}{\Delta x^2} + \frac{(y_1 - y_2)^2}{\Delta y^2}}$$

余弦相似度

$$C_{ij}(1) = \frac{\sum_{k=1}^n x_{ki} x_{kj}}{\left[\left(\sum_{k=1}^n x_{ki}^2 \right) \left(\sum_{k=1}^n x_{kj}^2 \right) \right]^{1/2}}, \quad i = 1, 2, \dots, p; j = 1, 2, \dots, p$$



- ① $\theta = 0$, $\cos \theta = \cos 0 = 1$ 一样, 完全相似 (躺 m/cn)
 - ② $0 < \theta < 90^\circ$, $\cos \theta \in (0, 1)$ 具有相似性 (躺解)
 - ③ $\theta = 90^\circ$, $\cos \theta = 0$, 独立 (不相关), 正交 (身向 婚姻)
 - ④ $90^\circ < \theta < 180^\circ$, $\cos \theta \in (-1, 0)$
 - ⑤ $\theta = 180^\circ$, $\cos \theta = -1$, 完全相反 (躺解)
- 语义空间相斥

点乘

$$\cos \theta = \frac{\vec{a} \cdot \vec{b}}{|\vec{a}| |\vec{b}|} = \frac{\vec{a} \cdot \vec{b}}{|\vec{a}| |\vec{b}|}$$

$$= \frac{x_1 x_2 + y_1 y_2}{\sqrt{x_1^2 + y_1^2} \sqrt{x_2^2 + y_2^2}}$$

3.整理K-means算法的原理和步骤 (Jp192)

9.2.1 K-means 原理和步骤

K-means 算法的工作原理：首先随机从数据集中选取 K 个点，每个点初始地代表每个簇的聚类中心^①，然后计算剩余各个样本到聚类中心的距离，将它赋给最近的簇^②，接着重新计算每一簇的平均值^③，^④整个过程不断重复，如果相邻两次调整没有明显变化，说明数据聚类形成的簇已经收敛。本算法的一个特点是在每次迭代中都要考察每个样本的分类是否正确。若不正确，就要调整，在全部样本调整完后，再修改聚类中心，进入下一次迭代。这个过程将不断重复直到满足某个终止条件，终止条件可以是以下任何一个：

- 1) 没有对象被重新分配给不同的聚类。
- 2) 聚类中心不再发生变化。
- 3) 误差平方和局部最小。

算法步骤：

- 1) 从 n 个数据对象中任意选择 k 个对象作为初始聚类中心；
- 2) 循环 3) 到 4) 直到每个聚类不再发生变化为止；
- 3) 根据每个聚类对象的均值（中心对象），计算每个对象与这些中心对象的距离；并根据最小距离重新对相应对象进行划分；
- 4) 重新计算每个聚类的均值（中心对象），直到聚类中心不再变化。这种划分使得下式最小：

$$E = \sum_{j=1}^k \sum_{x_i \in \omega_j} \|x_i - m_j\|^2$$

K-means 算法是很典型的基于距离的聚类算法，采用距离作为相似性的评价指标，即认为两个对象的距离越近，其相似度就越大。该算法认为簇是由距离靠近的对象组成，因此把得到紧凑且独立的簇作为最终目标。

K-means 算法：

输入：聚类个数 k，以及包含 n 个数据对象的数据库。

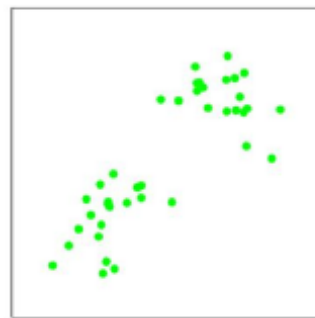
输出：满足方差最小标准的 k 个聚类。

K-means 的神奇之处：化静为动

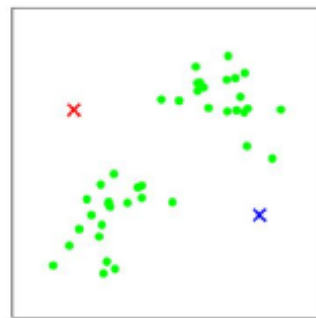
处理流程：

- 1) 从 n 个数据对象中任意选择 k 个对象作为初始聚类中心；
- 2) 根据每个聚类对象的均值（中心对象），计算每个对象与这些中心对象的距离；并根据最小距离重新对相应对象进行划分；
- 3) 重新计算每个（有变化）聚类的均值（中心对象）；
- 4) 循环 2) 到 3) 直到每个聚类不再发生变化为止。

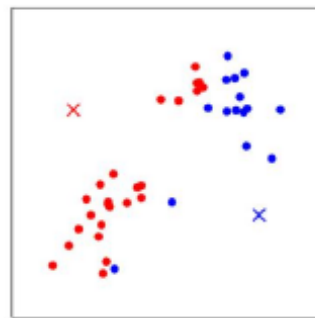
算法模拟图示



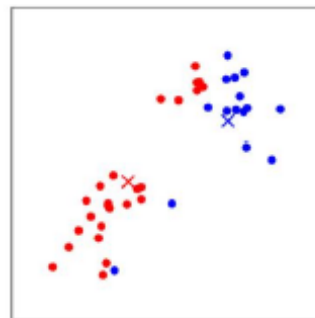
(a)



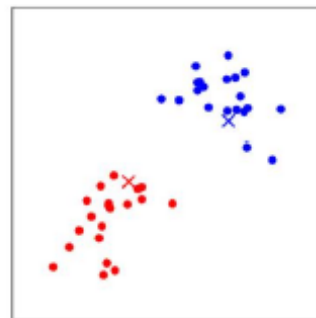
(b)



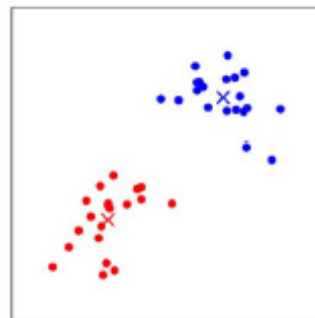
(c)



(d)



(e)



(f)

3.整理K-means算法的原理和步骤 (Jp192)

K-means算法的特点



在 K-means 算法中 K 是事先给定的, 这个 K 值的选定是非常难以估计的。(对样本的了解)



在 K-means 算法中, 首先需要根据初始聚类中心来确定一个初始划分, 然后对初始划分进行优化。



K-means算法需要不断地进行样本分类调整, 不断地计算调整后的新的聚类中心, 因此当数据量非常大时, 算法的时间开销是非常大的。(但是K-means依然是数据集较大时的优先选择, 主要在于其简洁、有效。)



K-means算法对一些离散点和初始K值敏感, 不同的距离初始值对同样的数据样本可能得到不同的结果。即K-means的结果是不稳定的, 即便在同样的K值情况下, 每次聚类的结果因着初始中心选择的不同, 出现不同的结果。



K-means无法做到将一个点分到不同的类中, 而一个对象存在类别交叉现象, 是很常见的。

9.2.4 K-means 特点

- ① 在 K-means 算法中 K 是事先给定的, 这个 K 值的选定是非常难以估计的。(对样本的了解)
- ② 在 K-means 算法中, 首先需要根据初始聚类中心来确定一个初始划分, 然后对初始划分进行优化。
- ③ K-means 算法需要不断地进行样本分类调整, 不断地计算调整后的新的聚类中心, 因此当数据量非常大时, 算法的时间开销是非常大的。(但 K-means 依然是数据集较大时的优先选择, 主要在于其简洁、有效)
- ④ K-means 算法对一些离散点和初始 K 值敏感, 不同的距离初始值对同样的数据样本可能得到不同的结果。(不稳定性)
- ⑤ K-means 无法做到将一个点分到不同的类中, 而一个对象存在类别交叉现象, 是很常见的。(重叠 K-means)

4. 整理HC算法的过程 (Jp198)

HC算法的原理

层次聚类算法是通过将数据组织为若干组并形成一个相应的树来进行聚类的。根据层次是自底向上还是自顶向下形成，层次聚类可以分为凝聚和分裂两种算法形式。

HC算法的步骤

- (1) 将D中每个样本点当作其类簇；
- (2) 循环
- (3) 找到分属两个不同类簇，且距离最近的样本点对；
- (4) 将两个类簇合并；
- (5) 直到类簇数=K。

输入: K 目标类目数; D 数据样本点集合;

输出: K个类簇集合

HC算法的特点 (小而美)

- (1) HC算法能够清晰的展示数据的层次结构。
- (2) HC算法本身是一个小而巧的算法，计算的复杂度很高，适用于比较小的数据集。

9.3 层次聚类 (Hierarchical Clustering, HC)

9.3.1 层次聚类原理和步骤

层次聚类算法，是通过将数据组织为若干组并形成一个相应的树来进行聚类的。根据层次是自底向上还是自顶向下形成，层次聚类算法可以进一步分为凝聚的聚类算法和分裂的聚类算法，如图 9-7 所示。一个完全层次聚类的质量由于无法对已经做的合并或分解进行调整而受到影响。但是层次聚类算法没有使用准则函数，它所含的对数据结构的假设更少，所以它的通用性更强。

在实际应用中一般有两种层次聚类方法：

1) 凝聚的层次聚类：这种自底向上的策略首先将每个对象作为一个簇，然后合并这些原子簇为越来越大的簇，直到所有的对象都在一个簇中，或者某个终结条件被达到要求。大部分的层次聚类方法都属于一类，它们在簇间的相似度的定义有点不一样。

2) 分裂的层次聚类：像这样的自顶向下的策略与凝聚的层次聚类有些不一样，它首先将所有对象放在一个簇中，然后慢慢地细分为越来越小的簇，直到每个对象自行形成一簇，或者直到满足其他的一个终结条件，例如满足了某个期望的簇数目，又或者两个最近的簇之间的距离达到了某一个阈值。

图 9-7 描述了一个凝聚的层次聚类方法 AGENES 和一个分裂的层次聚类方法 DIANA 在一个包括五个对象的数据的集合 {a,b,c,d,e} 上的处理过程。初始时，AGENES 将每个样本点自为一簇，之后这样的簇依照某一种准则逐渐合并，例如，例如簇 C1 中的某个样本点和簇 C2 中的一个样本点相隔的距离是所有不同类簇的样本点间欧几里得距离最近的，则认为簇 C1

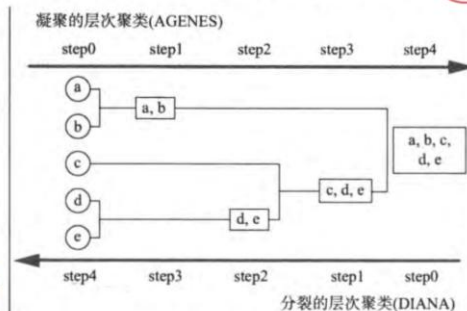
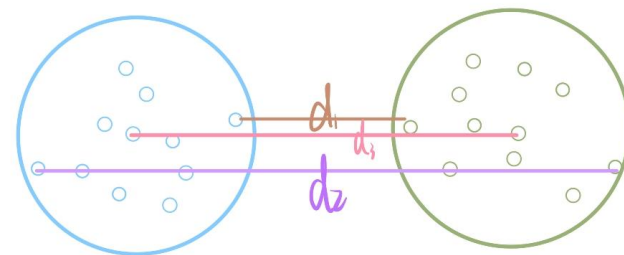


图 9-7 凝聚的层次聚类和分裂的层次聚类处理过程

簇间距离

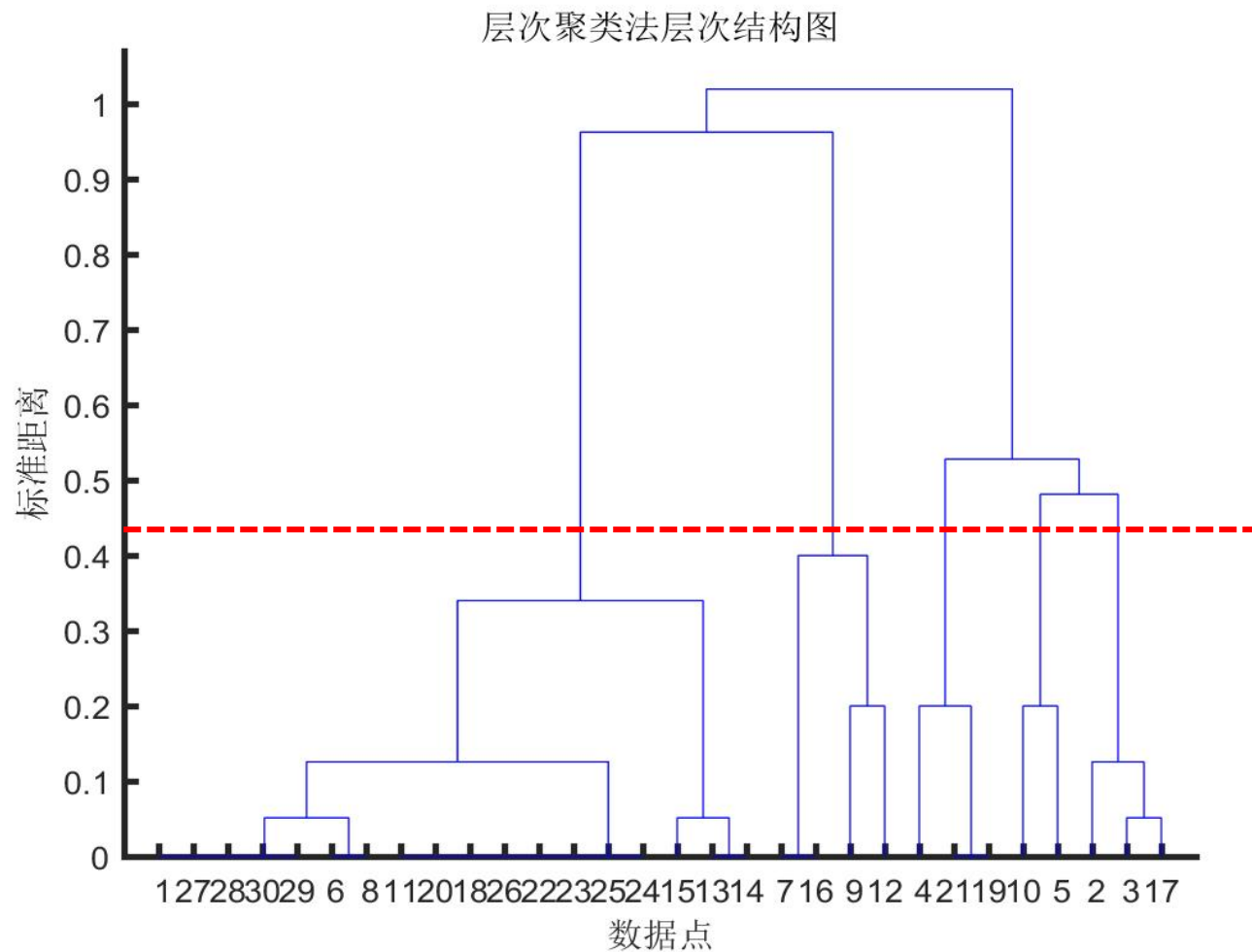
1. d_1 最近点
2. d_2 最远点
3. d_3 中心点



5.掌握PPT26页HC聚类结果的图（如果聚成5类，画一条直线）

HC聚类结果图

- 1、红线从上往下平移；
- 2、它与多少垂直线相交，就分成了几个类。
- 3、右图图示聚成5类。



6.阅读《白话机器学习》K-means章，整理读书笔记

阅读总结

本章通过聚类影迷性格特征这一示例引入对K均值聚类算法的介绍，指出K均值聚类的两个核心问题是：确定群组数量（K值），确定群组成员（群组中都有谁，即每个数据点如何聚类到相应的群组中。）

首先，针对确定群组数量问题，即是确定K值，教材向我们介绍了陡坡图这一工具，通过陡坡图的曲线拐点个数来确定群组数量较为合理。

紧接着，教材指出群组中的成员是在不断迭代过程中确定的，这与我们课上所学步骤一致：首先确定伪中心点。然后计算各个数据点到伪中心点的距离进行分配群组，而后再依据现有数据点调整伪中心点，重复上述调整步骤不断迭代直至最后群组成员不再发生变化方可完成K均值聚类。

最后，教材指出K均值聚类这一方法虽然简单朴素但仍然具有局限性，因此我们在实际使用过程中可以先用K均值聚类作初步了解数据结构，而后再运用更高级的算法深入分析。

第2章 K均值聚类

定义群组

群组特征

随着群组数量增加，每个群组中的成员彼此越来越相似，相邻群组之间的区别越来越不明显；

在极端情况下，每个数据点本身就是一个群组；（毫无意义）

群组数量

首先，群组数量要足够大，以便提取有意义的模式，用作商业决策参考；

其次，群组数量还要足够小，能够确保各个群组之间有明显的区别。

确定群组数量（陡坡图）

可以展现群组内散度随着群组数量增加而降低的过程；

若所有成员都属于同一个群组，则群组内散度将达到最大值。

陡坡图曲线的**拐点处**表示最佳群组数量，此处的群组内散度较为合理。

确定群组成员

预估每个群组的中心点。（伪中心点）

把每个数据点分配给最近的伪中心点。

根据群组成员的分布，调整伪中心点的位置。

重复步骤2、3，直至群组成员不再发生变化。

局限及改善方案

局限性

每个数据点只能属于一个群组。

群组被假定为正圆形的。

群组被假定为离散的。（非重叠）

改善方案

先用K均值聚类大致了解数据结构

再综合运用更高级的算法深入分析

6. 阅读《白话机器学习》K-means章，整理读书笔记

第2章 k 均值聚类

2.1 找出顾客群

让我们聊聊电影喜好这个话题。如果一个人喜欢《初恋 50 次》这部电影，那么他或她很可能也喜欢类似的“女性电影”，比如《27 套礼服》。这就是聚类的原理，即通过识别共同的喜好或特征，把顾客分组，以供零售商有针对性地投放广告。

然而，给顾客分组并非易事。我们可能一开始并不知道应该如何分组，也不知道应该分多少组。

k 均值聚类可以帮助我们回答这些问题。这个方法可以用来把顾客或产品分入不同的群组，其中 **k** 表示群组个数。

2.2 示例：影迷的性格特征

为了使用 **k 均值聚类** 方法找出顾客群，需要可量化的顾客信息。一个常用的变量是收入，因为与低收入顾客群相比，高收入顾客群往往更喜欢购买名牌商品。这样一来，商家就可以利用这些信息向高收入顾客群投放奢侈品广告。

性格特征是另一个常用的变量。在一项针对 Facebook 用户的研究中，研究人员邀请用户参与问卷调查，以了解他们在 4 种性格特征上的得分：外向型（对社会交往的喜欢程度）、尽责型（工作努力程度）、情绪型（受压力影响的程度）以及开放型（对新事物的接受程度）。

初步分析表明，这些性格特征之间存在正相关关系。高度尽责的人往往更外向，高度情绪化的人则往往更开放。因此，为了更好地对这些性格特征可视化，将它们两两配对——外向型和尽责型、情绪型和开放型——并统计每对的得分，然后在二维图中标出。

接下来，把每个人的总得分与他或她在 Facebook 上点赞的电影页面进行匹配。这样一来，就可以通过不同的性格特征给影迷分组，如图 2-1 所示。

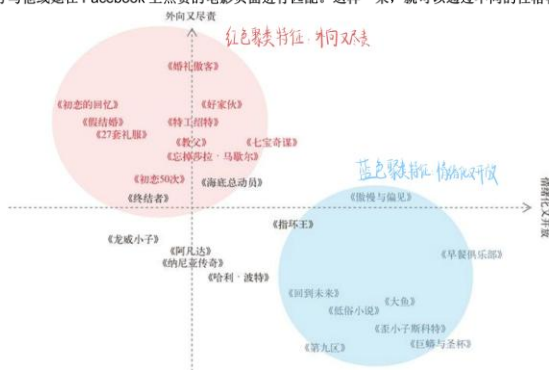


图 2-1 影迷的性格特征

在图 2-1 中，可以看到两个主要群组。

- 红色：外向又尽责的影迷，他们喜欢动作片和爱情片。
- 蓝色：情绪化又开放的影迷，他们喜欢先锋艺术片和奇幻片。

除了这两个群组外，中间部分的电影好像是大家都喜欢的。

根据这些信息，可以对广告投放进行规划。如果一个人喜欢《初恋 50 次》，那么就可以向他或她推荐一个群组中的其他电影，或者捆绑类似产品进行促销。

2.3 定义群组

在定义群组时，必须回答两个问题。

- 有多少个群组？
- 每个群组中有谁？

2.3.1 有多少个群组

这个问题很主观。虽然在图 2-1 中有两个群组，但它们可以被进一步划分。例如，蓝色群组可以被进一步划分成两个子群组：故事片（包括《傲慢与偏见》和《早餐俱乐部》）和奇幻片（包括《巨蟒与圣杯》和《亚小子斯科特》）。

随着群组数量增加，每个群组中的成员彼此越来越相似，相邻群组之间的区别则越来越不明显。在极端情况下，每个数据点本身就是一个群组，但这种分组方式毫无意义。

显然，在决定群组数量时必须有所权衡。首先，群组数量要足够大，以便提取有意义的模式，用作商业决策参考；其次，还要足够小，能够确保各个群组之间有明显的区别。

要确定合适的群组数量，一种方法是使用**陡坡图**，如图 2-2 所示。

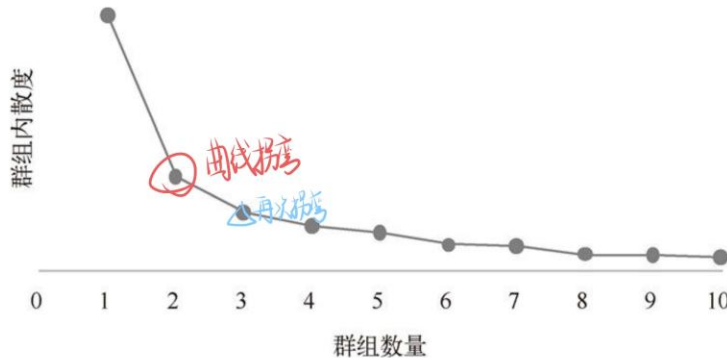


图 2-2 弯曲的陡坡图表明存在两三个群组

陡坡图可以展现群内散度，随群组数量增加而降低的过程。若所有成员都属于同一个群组，则群内散度将达到最大值。随着群组数量增加，各个群组变得更紧凑，群组成员也变得更相似。

陡坡图曲线的拐弯处表示最佳群组数量，此处的群内散度较为合理。从图 2-2 可以看到，当群组数量为 2 时，曲线拐弯，这两个群组对应于图 2-1 中的两个主要的电影群组。当群组数量为 3 时，曲线再次拐弯（尽管不如前一个明显），这意味着可以分出第 3 个群组，即普遍受欢迎的电影。但是，若继续增加群组数量，会导致群组变小，还会增大区分各个群组的难度。

确定好合适的群组数量之后，就该确定每个群组的成员了。

2.3.2 每个群组中有谁

群组成员是在迭代过程中确定的。下面以 2 个群组为例进行讲解，如图 2-3 所示。

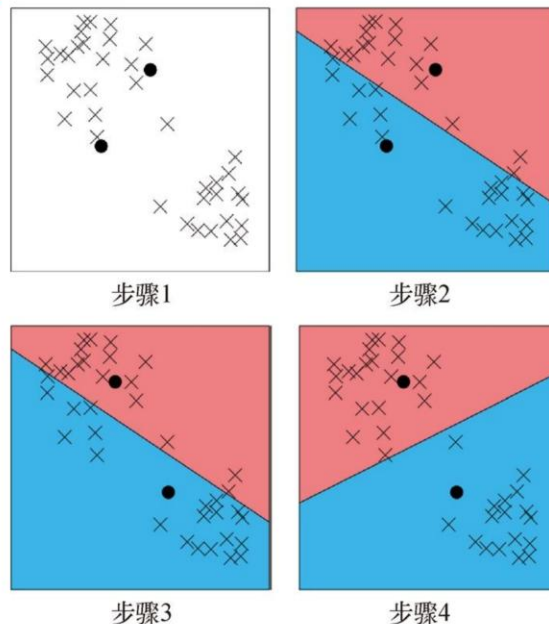


图 2-3 k 均值聚类方法的迭代过程

因为群组最好由密集的数据点组成，所以通过**检查群组成员与群组中心点的距离，便可以判断该群组的有效性**。不过，由于中心点的位置最初是未知的，因此只能**先假设**，然后把数据点分配给离它们最近的中心点。

接下来，不断调整中心点的位置，直至与真实的位置重合。而后，根据各数据点与中心点的距离，重新为每个数据点分配群组。如果某个数据点离原先所在群组的中心点较远，而离邻近群组的中心点较近，那就把它重新划入邻近的群组。**不断迭代、分配**

简单总结一下，为群组确定成员的过程包含如下步骤。不管群组数量有多少，这些步骤都适用。

步骤 1：首先猜测每个群组的中心点。因为暂时不能确定通过猜测得到的中心点是否正确，所以称它们为**伪中心点**。

步骤 2：把每个数据点分配给最近的伪中心点。这样一来，就得到了两个群组，即红色群组和蓝色群组，如图 2-3 所示。

步骤 3：根据群组成员的分布，调整伪中心点的位置。

步骤 4：重复步骤 2 和步骤 3，直至群组成员不再发生变化。

本例的分析只涉及 2 个维度。其实，聚类也可以在 3 个甚至更多的维度上进行。对于商家来说，更多的维度可能是顾客的年龄或到访的次数。虽然很难对多维分析进行可视化，但是可以借助程序计算数据点和群组中心点在多维情形下的距离。

2.4 局限性

尽管 **k 均值聚类** 方法很有用，但是它本身存在一定的局限性。

- **每个数据点只能属于一个群组。**然而，数据点可能恰好位于两个群组中间，无法通过 **k 均值聚类** 方法确定它应该属于哪个群组。
- **群组被假定是正圆形的。**查找距离某个群组中心点最近的数据点，这一迭代过程类似于缩小群组的半径，因此最终得到的群组在形状上类似于正圆形。假设群组的实际形状是椭圆形，那么在实际应用 **k 均值聚类** 方法之后，位于椭圆两端的数据点可能会被划入邻近的群组，这会造成很大的问题。
- **群组被假定是离散的。****k 均值聚类** 既不允许群组重叠，也不允许它们相互嵌套。

除了 **k 均值聚类** 之外，还有更可靠的聚类方法。这些聚类方法不会强制把每个数据点划入单个群组，而会**计算每个数据点属于其他群组的概率**，因此适合用来识别**非圆形或有重叠的群组**。

尽管 **k 均值聚类** 存在上述局限性，但是它的优点是简单朴素。一个好的数据分析策略是，先用 **k 均值聚类** 方法大致了解数据结构，再综合运用其他更高级的方法进行深入分析。这样做可以大大弥补 **k 均值聚类** 方法的局限性。

2.5 小结

- **k 均值聚类** 用于把相似的数据点划入同一个群组。群组数量 **必须事先指定**。
- 给数据点分组时，首先把各个数据点**分配到距离最近的群组中，然后调整群组中心点的位置，重复这两个步骤**，直到群组中的成员不再发生变化。
- **k 均值聚类** 最适合用于**正圆形、非重叠**的群组。

关联规则 课后作业5

 姓名：张苏蕊

 班级：信管2301

 学号：202321054007

1.整理资料：有关关联选股的背景知识

选股就是选公司，选公司就要先选行业。

好行业需要符合两个标准：

①市场需求要足够大，而且能长久。

②市场需求快速增长，甚至是爆发式增长。



消费、科技、医药

成长股的筛选

(1) 初步筛选：①毛利率30-40%以上；②股本越小，扩张动力越大，潜力也越大；③净资产收益率（ROE）5%以上，10%为佳；④主营业务占比越高越好，100%最好；⑤股票价格一般在10-15元以下；⑥行业壁垒；⑦需求；⑧股东结构；⑨投资非主营业务；

(2) 精细筛选：因人而异，因公司而异。一般先把主营业务看不懂的筛掉，其次把明显是靠关系的筛掉，最后会考虑一些纯粹的市场因素。

ROE指标筛选

ROE 反映的是股东每投入 1 块钱能够带来多少钱的利润。

1.整理资料：有关关联选股的背景知识

关联选股的核心原则

- 1、单个指标不可靠；
- 2、多个特征相互关联、同时满足时，胜率更高；
- 3、风险也往往是多个坏特征一起出现。

财务风险的负面关联规则（排雷）

存贷双高

商誉过高

关联交易

财务数据异常完美

毛利率远超同行

优质股关联

高毛利率

小盘股

高ROE

主营业务突出

有行业壁垒

你是如何从两千多只股票中选出心仪的股票的呢？

选股就是选公司，选公司就要先选行业。

因为行业是公司所在的赛道，是公司**生存的基础**，也决定了**公司的格局**。

所以我会从行业的角度出发，梳理出各优质赛道的主要玩家。

只要耐心看完，你就会知道未来**哪些行业的成功率更高**，它们的**底层逻辑是什么**，

在这些优质行业里**有哪些主要玩家**，它们的**商业模式、竞争优势**又是什么。

OK，我们这就发车。文章很长，干货很干，但全文没有多余的废话，请一定一定，耐心读完。

以下是目录：

第一部分：长牛黄金三赛道

- 1、消费行业
- 2、科技行业
- 3、医疗行业+

第二部分：风险排雷

- 1) 不符合常识
- 2) 财务数据过于优秀
- 3) 关联交易
- 4) 存贷双高
- 5) 商誉过高

第三部分：选股的注意事项

2、分析数据（清楚说明矩阵行与列的含义）

	I1	I2	I3	I4	I5
T001	1	1	0	0	1
T002	0	1	0	1	0
T003	0	1	1	0	0
T004	1	1	0	1	0
T005	1	0	1	0	0
T006	0	1	1	0	0
T007	1	0	1	0	0
T008	1	1	1	0	1
T009	1	1	1	0	0

行的含义

每一行对应一个独立的事务记录，通常来自超市购物篮、订单、日志记录等。

列的含义

代表项，每一列对应一个具体的商品、属性或事件。

矩阵元素含义

矩阵中每个单元格的元素为布尔值(0/1)

1:表示该列对应的项目，出现在该行对应的事务中(如T100行I1列=1,代表事务T100包含项目I1)。

0:表示该列对应的项目，未出现在该行对应的事务中(如 T100行I4列=0,代表事务T100不包含项目I4)。

3、运行程序（使用python，参考压缩文件包中的程序）

jupyter

生成表格数据

Last Checkpoint: 11 months ago

File

Edit

View

Run

Kernel

Settings

Help

+

✂

📄

📄

▶

■

🔄

▶▶

Code

▼

[1]:

```
import csv
import random

# 生成商品池 (示例商品名称)
products = [
    "智能手机", "蓝牙耳机", "机械键盘", "智能手表", "平板电脑",
    "无线鼠标", "笔记本电脑", "显示器", "移动电源", "摄像头",
    "游戏手柄", "投影仪", "打印机", "路由器", "硬盘",
    "电动牙刷", "吹风机", "空气炸锅", "咖啡机", "吸尘器",
    "加湿器", "电风扇", "电饭煲", "微波炉", "烤箱"
]

# 生成数据行
data = []
for i in range(1, 101):
    selected = random.sample(products, 3) # 确保同一行三个不重复
    data.append([i, selected[0], selected[1], selected[2]])

# 写入CSV文件
with open('products.csv', 'w', newline='', encoding='utf-8-sig') as f:
    writer = csv.writer(f)
    writer.writerow(["序号", "商品1", "商品2", "商品3"])
    writer.writerows(data)

print("CSV文件已生成: products.csv")
```

CSV文件已生成: products.csv

[]:

[]:

Delimiter:

,

	序号	商品1	商品2	商品3
1	1	移动电源	吸尘器	电动牙刷
2	2	路由器	硬盘	摄像头
3	3	路由器	硬盘	电风扇
4	4	智能手机	机械键盘	平板电脑
5	5	蓝牙耳机	机械键盘	吸尘器
6	6	路由器	蓝牙耳机	微波炉
7	7	投影仪	平板电脑	电动牙刷
8	8	微波炉	吹风机	加湿器
9	9	烤箱	机械键盘	投影仪
10	10	电风扇	移动电源	空气炸锅
11	11	笔记本电脑	移动电源	投影仪
12	12	游戏手柄	智能手表	智能手机
13	13	游戏手柄	智能手机	机械键盘
14	14	投影仪	蓝牙耳机	微波炉
15	15	蓝牙耳机	智能手机	电饭煲
16	16	电饭煲	智能手表	移动电源
17	17	电饭煲	无线鼠标	硬盘
18	18	移动电源	咖啡机	吸尘器
19	19	蓝牙耳机	智能手机	机械键盘
20	20	蓝牙耳机	游戏手柄	硬盘
21	21	平板电脑	吸尘器	打印机
22	22	无线鼠标	投影仪	游戏手柄
23	23	加湿器	吸尘器	电风扇
24	24	笔记本电脑	烤箱	移动电源
25	25	机械键盘	咖啡机	无线鼠标
26	26	显示器	笔记本电脑	平板电脑
27	27	打印机	投影仪	蓝牙耳机

3、运行程序（使用python，参考压缩文件包中的程序）

jupyter 使用apriori构建商品知识图谱 Last Checkpoint: 11 months ago

File Edit View Run Kernel Settings Help

Code

```
[3]: """ 步骤3: 数据准备
# 读取CSV文件
df = pd.read_csv('products.csv')

# 将每行转换为商品列表
transactions = df[['商品1', '商品2', '商品3']].values.tolist()

# 转换事务数据格式
te = TransactionEncoder()
te_ary = te.fit(transactions).transform(transactions)
trans_df = pd.DataFrame(te_ary, columns=te.columns_)
```

[4]: trans_df

	加湿器	吸尘器	吹风机	咖啡机	平板电脑	微波炉	打印机	投影仪	摄像头	无线鼠标	...	烤箱	电动牙刷	电风扇	电饭煲	硬盘	移动电源	空气炸锅	笔记本电脑	蓝牙耳机	路由器
0	False	True	False	False	False	False	False	False	False	False	...	False	True	False	False	False	True	False	False	False	False
1	False	False	False	False	False	False	False	False	True	False	...	False	False	False	False	True	False	False	False	False	True
2	False	False	False	False	False	False	False	False	False	False	...	False	False	True	False	True	False	False	False	False	True
3	False	False	False	False	True	False	False	False	False	False	...	False	False	False	False	False	False	False	False	False	False
4	False	True	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False	False	True	False
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
95	False	False	False	False	False	False	False	False	False	False	...	False	True	False	False	True	False	False	False	False	False
96	False	False	True	False	True	False	False	False	False	False	...	False	False	False	True	False	False	False	False	False	False
97	True	True	False	False	False	False	False	False	False	False	...	True	False	False	False	False	False	False	False	False	False
98	False	False	False	False	False	True	False	False	False	True	...	False	False	True	False	False	False	False	False	False	False
99	False	False	False	False	True	False	False	False	False	False	...	False	False	False	False	False	False	False	False	False	True

100 rows × 25 columns

jupyter 使用apriori构建商品知识图谱 Last Checkpoint: 11 months ago

File Edit View Run Kernel Settings Help

Code

```
[5]: """ 步骤4: 应用Apriori算法
# 计算频繁项集 (设置最小支持度0.02, 即至少出现2次)
frequent_itemsets = apriori(trans_df,
                             min_support=0.02,
                             use_colnames=True,
                             max_len=2) # 只需要到2项集

# 转换项集格式
frequent_itemsets['itemsets'] = frequent_itemsets['itemsets'].apply(lambda x: list(x))
frequent_itemsets['length'] = frequent_itemsets['itemsets'].apply(len)
```

```
[6]: """ 步骤5: 过滤结果 (保留出现2次以上的2项集)
# 计算实际出现次数
frequent_itemsets['count'] = (frequent_itemsets['support'] * len(transactions)).astype(int)
qualified = frequent_itemsets[(frequent_itemsets['length'] == 2) & (frequent_itemsets['count'] >= 2)]
```

[7]: qualified

	support	itemsets	length	count
25	0.03	[吸尘器, 加湿器]	2	3
26	0.02	[显示器, 加湿器]	2	2
27	0.02	[烤箱, 加湿器]	2	2
28	0.02	[电风扇, 加湿器]	2	2
29	0.02	[电饭煲, 加湿器]	2	2
...	...	...	...	...
90	0.02	[电饭煲, 蓝牙耳机]	2	2
91	0.02	[硬盘, 路由器]	2	2
92	0.03	[空气炸锅, 移动电源]	2	3
93	0.02	[笔记本电脑, 移动电源]	2	2
94	0.02	[笔记本电脑, 蓝牙耳机]	2	2

70 rows × 4 columns

3、运行程序（使用python，参考压缩文件包中的程序）

jupyter 使用apriori构建商品知识图谱 Last Checkpoint: 11 months ago

File Edit View Run Kernel Settings Help

Code

```
G = nx.Graph()

# 添加节点（所有出现过的商品）
all_products = list(te.columns_)
G.add_nodes_from(all_products)

# 添加边（关联关系）
for _, row in qualified.iterrows():
    items = list(row['itemsets'])
    G.add_edge(items[0], items[1], weight=row['count'], count=row['count'])
```

```
[10]: #%% 步骤7: 可视化知识图谱
plt.figure(figsize=(14, 10))

# 设置布局
pos = nx.spring_layout(G, k=0.5)

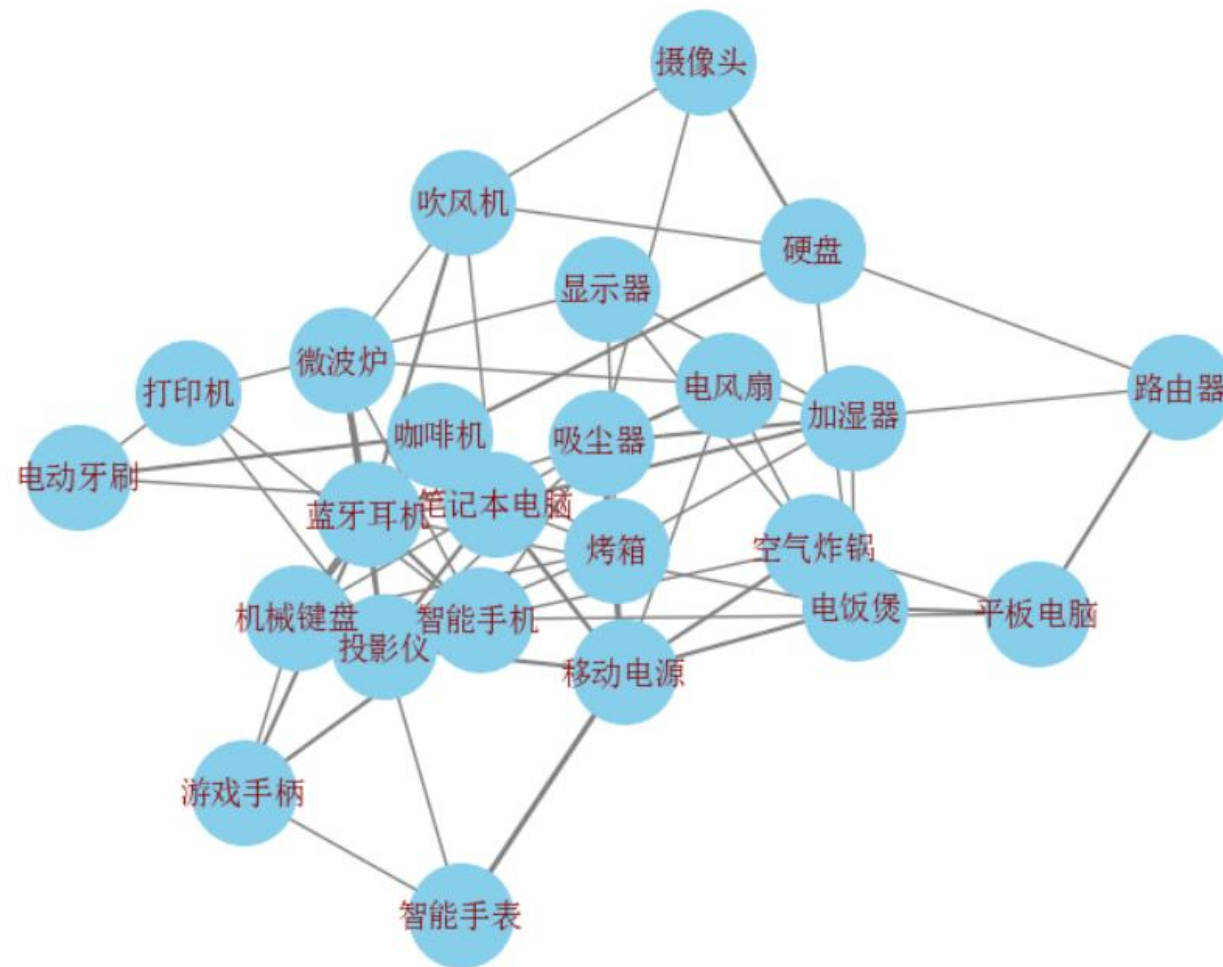
# 绘制节点
nx.draw_networkx_nodes(G, pos, node_size=1500, node_color='skyblue')

# 绘制边
edges = G.edges(data=True)
nx.draw_networkx_edges(G, pos, edgelist=edges,
                        width=[d['weight']*0.5 for (u,v,d) in edges],
                        edge_color='gray')

# 绘制标签
nx.draw_networkx_labels(G, pos, font_size=12,
                        font_family='SimSun',
                        font_color='darkred')

# 显示图表
plt.title("商品关联分析",
        fontdict={
            'fontfamily': 'SimHei', # macOS内置支持中文的字体
            'fontsize': 16,
            'fontweight': 'bold',
            'color': 'navy'
        })
```

商品关联分析



4、分析结果，思考教材分析过程的含义（并指出教材中的错误）。

第二次扫描：

为发现频繁 2 项集的集合 L2，算法使用 $L1 \bowtie L1$ 算法产生候选 2 项集的集合 C2，然后再根据支持度得到 L2：

C1	C2	L2
项集	项集支持度计数	项集支持度计数
(I1,I2)	(I1,I2) 4	(I1,I2) 4
(I1,I3)	(I1,I3) 4	(I1,I3) 4
(I1,I4)	(I1,I4) 1	(I1,I5) 2
(I1,I5)	(I1,I5) 2	(I2,I3) 4
(I2,I3)	(I2,I3) 4	(I2,I4) 2
(I2,I4)	(I2,I4) 2	(I2,I5) 2
(I2,I5)	(I2,I5) 2	
(I3,I4)	(I3,I4) 0	
(I3,I5)	(I3,I5) 1	
(I4,I5)	(I4,I5) 0	

第三次扫描：**剪枝**

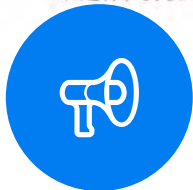
$L2 \bowtie L2$ 产生候选 3 项集的集合 C3。

候选 3 项集 C3 的产生详细地列表如下：

1) 连接 $C3 = L2 \bowtie L2 = \{(I1,I2), \{I1,I3\}, \{I1,I5\}, \{I2,I3\}, \{I2,I4\}, \{I2,I5\}\} \bowtie \{(I1,I2), \{I1,I3\}, \{I1,I5\}, \{I2,I3\}, \{I2,I4\}, \{I2,I5\}\} = \{(I1,I2,I3), \{I1,I2,I5\}, \{I1,I3,I5\}, \{I2,I3,I4\}, \{I2,I3,I5\}, \{I2,I4,I5\}\}$

2) 使用 Apriori 性质剪枝：频繁项集的所有非空子集也必须是频繁的。例如， $\{I1,I3,I5\}$ 的 2 项子集是 $\{I1,I3\}$ 、 $\{I1,I5\}$ 和 $\{I3,I5\}$ 。 $\{I3,I5\}$ 不是 L2 的元素，因而不是频繁的。因此，从 C3 中删除 $\{I1,I3,I5\}$ 。

3) 这样，剪枝 $C3 = \{(I1,I2,I3), \{I1,I2,I5\}\}$ 。



结果分析

- 1、商品间存在频繁共现组合。
- 2、数码、家电类商品关联性较强，常被同时购买，形成明显的关联群体。
- 3、部分商品购买行为独立，与其他商品关联度较低。

教材分析过程的含义

- 1、通过事务矩阵、频繁项集、强关联规则来挖掘行业、热点、个股之间的联动规律，为关联选股提供算法支撑。
- 2、其过程主要是将市场关联量化为数据，通过逐层搜索和剪枝，提取稳定有效的投资逻辑，实现选股的自动化、科学化。

教材错误

- 1、第二次扫描应该产生候选集 C2 ；
- 2、第三次扫描的连接候选集 C3 不完整。



简单线性回归模型 课后作业6

 姓名：张苏蕊

 班级：信管2301

 学号：202321054007

1、为什么X必须是列向量？变成行向量会怎么样呢？

概念阐释

行向量：(1, N) $\rightarrow$ 1 行很多列

列向量：(N, 1) $\rightarrow$ 1 列很多行

数学本质（矩阵乘法）

线性回归数学本质是： $y = X \cdot w + b$

矩阵乘法规则：

前一个矩阵的列数 = 后一个矩阵的行数

X：输入的数据（二维矩阵），如果有M行，N个变量（属性/特征的话），那么这个矩阵就是：M $\times$ N，该案例中为(25, 1)的矩阵。

W：权重（斜率），该案例中为(1, 1)的矩阵；

矩阵乘法：(25, 1) $\times$ (1, 1) $\rightarrow$ 可以顺利计算。

模型原理

sklearn 模型中特征 X 应是二维数组：(样本数M, 特征数N)。
该案例中只有 1 个特征（时长Hours），样本数25个，所以必须是 (25,1) 的列向量。

变成行向量？

1、矩阵乘法：

(1, 25) $\times$ (1, 1) $\rightarrow$ 列数 $\neq$ 行数，模型从数学矩阵乘法上就无法完成计算。

2、模型会认为该案例中只有1个样本、25个特征，但实际是 25个样本、1个特征。
数据结构对不上，模型将无法成功训练。

2、如何显示生成的“熟模型”？ $y = b_0 + b_1 * x$, b_0 和 b_1 怎么获得？

概念阐释

一元线性回归： $y = b_0 + b_1 * x$

b_0 : 截距（直线与 y 轴的交点， $x=0$ 时的预测值）

b_1 : 斜率（系数， x 每增加 1， y 增加多少）

对应代码

```
b1 = regressor.coef_
```

输出：array([9.91065648]) 即斜率约等于9.91。

```
b0 = regressor.intercept_
```

输出：np.float64(2.0181600414346974) 即截距约等于2.02。

最终模型： $y = 2.02 + 9.91x$

代码解释

1、regressor.coef_

① 存储的是所有特征的斜率（系数）；

② 该案例中，特征为1，所以coef_是一个长度为1的数组 [9.91065648]，即 b_1 。

③ 若为多元线性回归，coef_会是 [$b_1, b_2, \dots$]，对应多个特征的斜率。

2、regressor.intercept_

① 是模型自动计算的截距 b_0 ，是一个单独的浮点数（不是数组）；

② 表示当 $x=0$ 时， y 的预测值。

3、如何评价结果的好坏？（代码） R^2 的结果是什么？

概念阐释

- 1、评价好坏的核心指标： R^2 （决定系数）
- 2、 R^2 作用：衡量模型对数据的解释能力有多强。
- 3、取值范围：0 ~ 1；越接近1，模型越好；越接近0，模型越差。

对应代码

```
1、R2_train = regressor.score(X_train,  
Y_train)
```

输出：0.9515510725211552

```
2、R2_test = regressor.score(X_test,  
Y_test)
```

输出：0.9454906892105355

数学本质

$$R^2 = 1 - \frac{\sum_{i=1}^n (y - \hat{y})^2}{\sum_{i=1}^n (y - \bar{y})^2}$$

分子：残差平方和（模型预测值和真实值的误差，越小越好）

分母：总平方和（真实值和平均值的误差，固定不变）

R^2 越接近 1，说明模型的误差越小，对数据的解释能力越强。

结果解读

- 1、模型在训练数据上，能解释95.16%的分数波动，拟合效果好，学习时长和分数的线性关系强；
- 2、模型在测试数据上，能解释94.55%的分数波动，泛化能力强。没有过拟合，在新数据上的表现和训练数据几乎一致，较稳定。

4、调整训练/测试的比例，对结果 (R^2) 有什么影响？试试看。

```
File Edit View Run Kernel Settings Help Trusted
+ ✂ 📄 📁 ▶ ■ ↺ ⏩ Code ▾ JupyterLab Python 3 (ipykernel) Anaconda Toolbox

[6]: (25, 1)

[7]: print(type(X))
<class 'numpy.ndarray'>

[8]: Y = dataset.iloc[:,1].values

[9]: Y
array([21, 47, 27, 75, 30, 20, 88, 60, 81, 25, 85, 62, 41, 42, 17, 95, 30,
       24, 67, 69, 30, 54, 35, 76, 86])

[10]: # 第2步 训练集，使用简单线性回归模型来训练

[11]: from sklearn.model_selection import train_test_split

[12]: X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size = 0.1, random_state = 0) # random_state = 0 每次运行分配可能不一样。=1就是一样的

[13]: from sklearn.linear_model import LinearRegression # 从Linear_model库中，调出LinearRegression类

[14]: regressor = LinearRegression() # 创建regressor对象。我们称为生模型

[15]: regressor
LinearRegression
  Parameters

[16]: # 使用划分训练集、测试集的方法来处理线性回归问题

[17]: regressor = regressor.fit(X_train, Y_train) # .fit # 训练之后，就成了熟模型 # 问“X_train 多少行？”

[18]: regressor
LinearRegression
  Parameters
```

分别修改比例为0.1、0.2、0.3、0.4、0.5

4、调整训练/测试的比例，对结果 (R^2) 有什么影响？试试看。

测试集比例	训练集 R^2	测试集 R^2
10%	0.9530	0.9298
20%（原比例）	0.9516	0.9455
30%	0.9485	0.9568
40%	0.9440	0.9566
50%	0.9569	0.9364

1、测试集比例过小（10%）

测试集样本太少，导致测试集 R^2 偏低，评估结果不稳定，无法真实反映模型的泛化能力。

2、测试集比例适中（20%-40%）

- ①训练集 R^2 随测试集比例增大缓慢下降，模型对训练集的拟合程度轻微降低；
- ②测试集 R^2 先上升后保持降低，在30%-40%最优，即评估最可靠、泛化能力最强。

3、测试集比例过大（50%）

- ①训练集数据太少，模型学习不充分，出现过拟合现象。
- ②训练集 R^2 升高，测试集 R^2 下降，模型泛化能力变差。

5、采用“最小二乘法”实现该模型

Jupyter 最小二乘法 Last Checkpoint: 23 minutes ago

File Edit View Run Kernel Settings Help

Code

```
[2]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
```

[3]: # 1. 数据预处理

```
dataset = pd.read_csv('studentscores.csv')
X = dataset.iloc[:, :1].values
Y = dataset.iloc[:, 1].values
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=0)
```

[4]: # 2. 最小二乘法求解

```
x_mean = np.mean(X_train)
y_mean = np.mean(Y_train)
numerator = np.sum((X_train - x_mean) * (Y_train.reshape(-1,1) - y_mean))
denominator = np.sum((X_train - x_mean) ** 2)
w = numerator / denominator
b = y_mean - w * x_mean
```

```
print("斜率 w =", round(w,2))
print("截距 b =", round(b,2))
print(f"回归方程: y = {w:.2f}x + {b:.2f}")
print(f"最小二乘法求解结果: 斜率 w = {w:.4f}, 截距 b = {b:.4f}")
```

```
斜率 w = 9.91
截距 b = 2.02
回归方程: y = 9.91x + 2.02
最小二乘法求解结果: 斜率 w = 9.9107, 截距 b = 2.0182
```

[5]: #3. 计算 R^2 决定系数 (评估拟合优度)

```
Y_train_pred = w * X_train + b
Y_train_2d = Y_train.reshape(-1, 1)

y_train_mean = np.mean(Y_train)
ss_tot = np.sum((Y_train_2d - y_train_mean) ** 2)
ss_res = np.sum((Y_train_2d - Y_train_pred) ** 2)
r2 = 1 - (ss_res / ss_tot)

print(f" $R^2$  = {r2:.4f}")
```

```
 $R^2$  = 0.9516
```

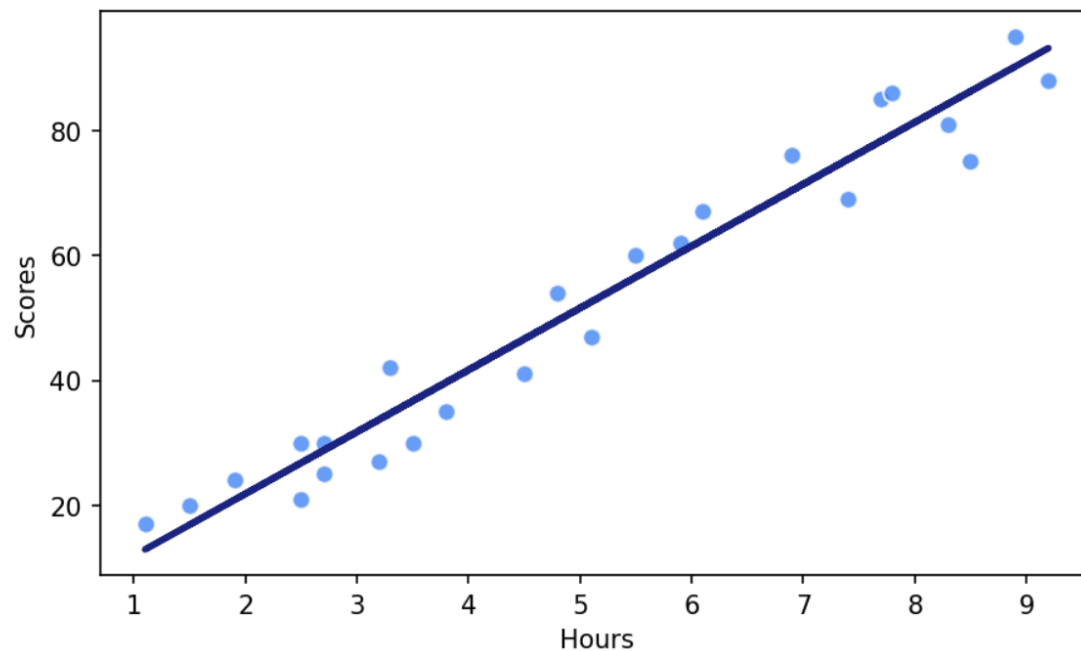
Jupyter 最小二乘法 Last Checkpoint: 25 minutes ago

File Edit View Run Kernel Settings Help

Code

[8]: # 4. 可视化

```
Y_pred_manual = w * X + b
plt.figure(figsize=(7,4), dpi=150)
plt.scatter(X, Y, color='#4285F4', alpha=0.8, s=50, edgecolor='white')
plt.plot(X, Y_pred_manual, color='#1A237E', linewidth=2.5)
plt.xlabel('Hours')
plt.ylabel('Scores')
plt.show()
```





多元线性回归模型 课后作业7

 姓名：张苏芯

 班级：信管2301

 学号：202321054007

1、观察数据集的特征，有几个属性？属性中的分类变量是什么？可视化分类变量（城市）和因变量（营收）之间的关系。

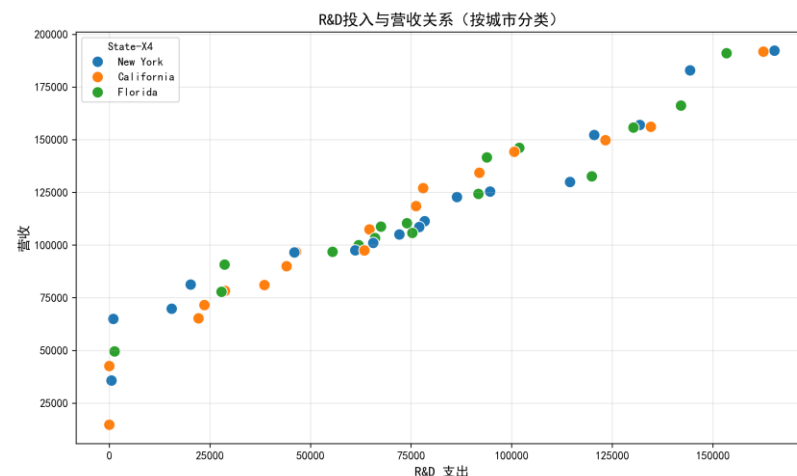
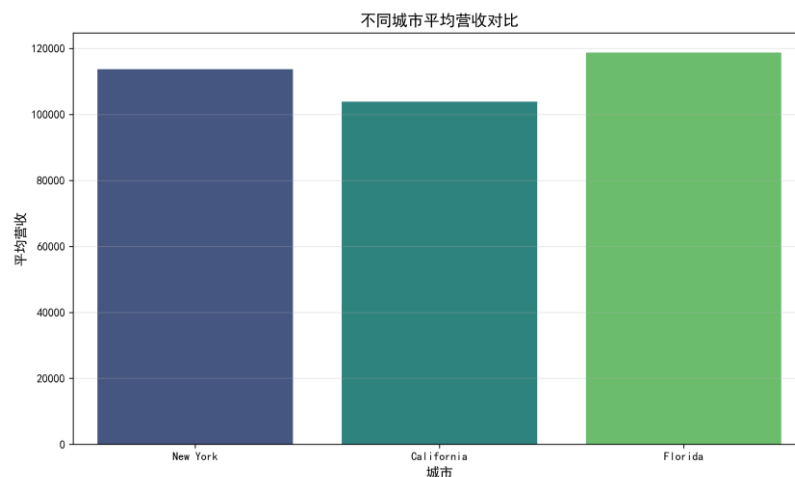
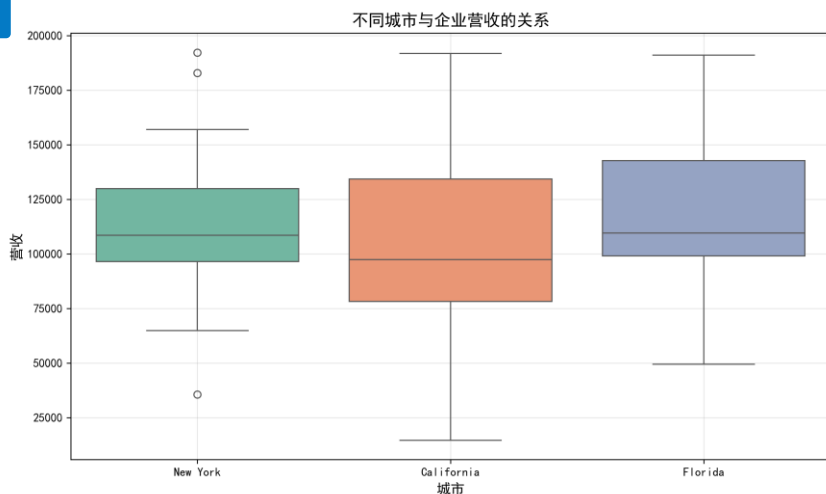
数据集特征

属性个数：5个

属性：R&D Spend、Administration、Marketing Spend、State、Profit（因变量）

分类变量：State（州），取值：California、Florida、New York

可视化分类变量（城市）和因变量（营收）之间的关系



展示了纽约、加州、佛罗里达三城企业营收分布。纽约营收中位数最高且分布集中，加州分布跨度最大、分化明显，佛罗里达整体水平居中且无极端异常值。

对比三城企业平均营收，佛罗里达最高，纽约次之，加州最低。佛罗里达整体盈利水平最优，加州受低营收样本拖累，均值表现偏弱。

显示 R&D 投入与营收呈强正相关，且三城样本高度重合，无明显区域分隔。说明研发投入是营收核心驱动因素，城市差异仅为次要影响。

2、如何把分类变量转化为onehot, 过程是什么？

2、创建编码器对象

使用 ColumnTransformer 构建转换器，指定只对分类变量所在列进行编码，其他数值列保持不变。

4、执行编码转换

调用 fit_transform 对数据集进行编码，将 3 个城市 (California、Florida、New York) 转化为 3 列由 0 和 1 组成的数值特征。

1、导入所需工具包

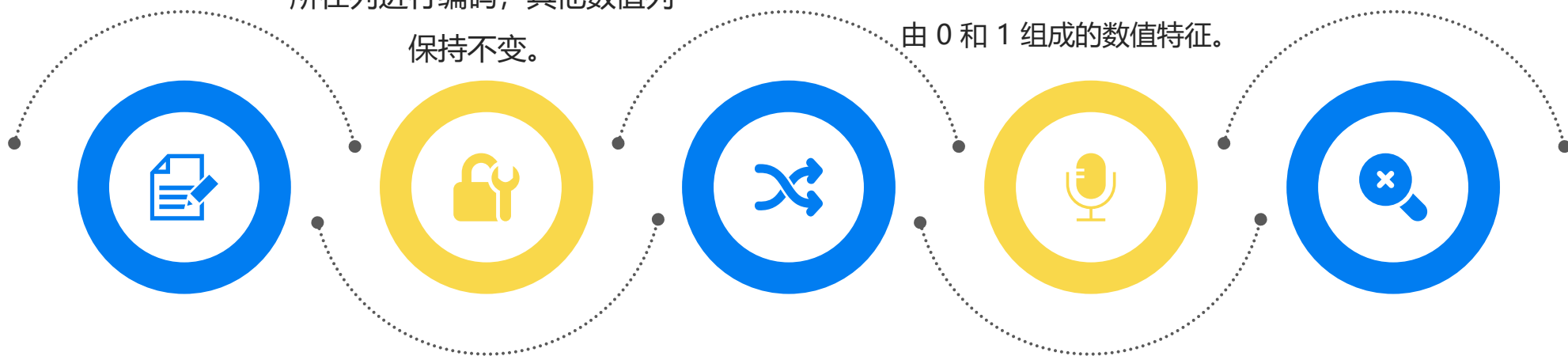
从 sklearn 库中导入 OneHotEncoder (独热编码器) 和 ColumnTransformer (列转换器)。

3、指定编码列

代码中指定对 *索引为 3 的列 (State-X4 城市列) 应用 OneHotEncoder。

5、避免虚拟变量陷阱

编码后会产生冗余特征，因此使用切片 $X[:, 3:6]$ 删除第一列编码结果，只保留 2 列虚拟变量，防止多重共线性，保证回归模型正常运行。



2、如何把分类变量转化为onehot，过程是什么？

转化代码

```
[9]: from sklearn.preprocessing import LabelEncoder, OneHotEncoder
      from sklearn.compose import ColumnTransformer
      labelencoder_X = LabelEncoder() # 先用LabelEncoder把分类变量标签化，就是用数字标号代替文字
      X[:, 3] = labelencoder_X.fit_transform(X[:, 3])
```

```
[10]: X[0:10]
```

```
array([[165349.2, 136897.8, 471784.1, 2], ...])
```

```
[11]: ct = ColumnTransformer(transformers=[('encoder', OneHotEncoder(), [3])], remainder='passthrough')
      X = np.array(ct.fit_transform(X))
      ***由于现在的onhotencoder会默认对整个数据集进行处理，并且没有了categorical_features，所以利用ColumnTransformer来限定对第一列进行编码***
      ***其实也可以通过pandas中的get_dummies()来进行onehot编码***
```

```
[12]: X
```

```
array([[0.0, 0.0, 1.0, 165349.2, 136897.8, 471784.1], ...])
```

```
[13]: ## 回避虚拟变量问题
```

```
[14]: X = X[:, 3:6]
```

```
[15]: X
```

```
array([[165349.2, 136897.8, 471784.1], ...])
```

3、如何评估结果？ R^2

核心概念

- 1、评估多元线性回归模型拟合效果的核心指标： R^2 （决定系数）
- 2、 R^2 作用：衡量模型对数据的解释能力有多强。
- 3、取值范围：0 ~ 1；越接近1，模型越好；越接近0，模型越差。

代码实现

第4步 评估系数 R^2

```
[21]: R2 = regressor.score(X,Y)  
      R2
```

```
[21]: 0.9490884301964866
```

结果解读

1、数值含义：

$R^2=0.949$ 表示该模型可以解释94.9%的企业营收变异。

2、结论

该模型的拟合效果较好，说明模型所选取的自变量能很好地预测企业营收，模型的预测能力强、可靠性高。

5、再次思考本次案例采用数据的特征，分类变量的存在，对多元线性回归带来怎样的影响？

分类变量对多元线性回归的影响

- 1、文字无法计算：模型只识别数值，不能直接处理 “New York、California、Florida” 这类文字。而原始文本为非数值类型，无法参与数学运算，不能直接输入回归模型。
- 2、数字编码会带来错误顺序：如果把城市强行标为 1、2、3，模型会误以为它们有大小、高低顺序，而城市只是无序分类，没有大小关系，会造成结果严重偏差。
- 3、需用 **One-Hot** 编码：One-Hot 编码将分类变量转为 0/1 数值特征，既变成数值，又不引入错误的大小关系，保证回归结果正确。

6、能否把分类变量直接lable化，也就是变成0、1、2后直接计算？不转化称one-hot合理吗？

不合理及其原因

1、**变量属性问题：**不能直接标签化为 0、1、2 直接计算。城市属于无序分类变量，无高低、大小、先后等级关系。

2、**虚假顺序问题：**若直接用 0、1、2 数字赋值，模型会错误判定分类之间存在数值大小与递进关系，人为制造不存在的数量关联。

3、**模型误差问题：**不做 One-Hot 编码直接运算，会导致回归系数偏移、变量权重失真，模型分析结果不合理，回归失去准确性与实际意义。

总结：不合理。无序分类变量禁止直接 Label 编码代入回归，必须使用 One-Hot 编码。



KNN算法 课后作业8

 姓名：张苏蕊

 班级：信管2301

 学号：202321054007

1、加入性别因素（进行one-hot编码），看看结果Acc值有没有变化？

未添加性别因素Acc值

添加性别因素后Acc值

生成混淆矩阵

- 我们怎么知道测试结果的好坏呢？
- 使用的工具，就是混淆矩阵。这个东西听起来很吓人，其实很简单
- 你想想看，我们的数据，或者客户，被分成了0或者1，如果“熟模型”，可以把测试集里面所有的0都正确判断为0，所有的1都正确判断为1，那么这个模型就非常厉害，它的准确率就是100%
- 但是，熟模型，作为一个机器，它是会犯错的。通俗的讲，它会糊涂，它会搞混淆，把本该是0判断成1，把本该是1判断成0
- 那么，它到底把多少真0判断成假1，把多少真1判断成假0，我们就用一个统计的工具“混淆矩阵”，来罗列出来
- 这就是混淆矩阵。就是判断模型的混淆程度，或者糊涂程度

```
[29]: from sklearn.metrics import confusion_matrix # 这个工具也是现成的
```

```
[31]: cm = confusion_matrix(y_test, y_pred) # y_test就是测试集原来的标签（人给的）；y_pred就是熟模型给的，他们两个的差异，就是混淆矩阵
```

- 就长下面这个样子
- 简单来说，对角线上的，就是全部判断对的，所有4个值加起来，就是测试数据总个数100
- 64对应的是 (0, 0)，0全部判断为0；4代表，有4个应该是0判断成1；3代表，有3个应该是1判断成0；29代表，1判断对了的个数
- 这样就是100个当中，7个判断错了，93个判断对了

```
[32]: cm
```

```
array([[55,  3],
       [ 1, 21]])
```

- 关于混淆矩阵，更加详细的内容，我特别建议大家参考下面的博客，这样可以了解的清楚一点
- 博客地址：<https://www.jianshu.com/p/27228ad417d4>

```
[33]: accuracy = (cm[0,0]+cm[1,1])/(cm[0,0]+cm[0,1]+cm[1,0]+cm[1,1]) # 用对角线上的和，除以总数，就是准确率。准确率93%，还真不错呢！
```

```
[34]: accuracy
```

```
[35]: 0.95
```

变化分析

在加入性别特征并完成编码后，模型的预测准确率没有发生明显变化，说明性别在本次数据中对购买行为的影响很小，不是关键区分因素。

```
[31]: # 1 加入性别因素
```

```
[32]: import pandas as pd
import numpy as np # 一个熟悉的库，numpy
```

```
[33]: dataset = pd.read_csv('Social_Network_Ads.csv')
dataset.head()
```

```
<div>***
```

```
[34]: # 划分 X 和 y
```

```
[35]: X = dataset.iloc[:,1:4].values
y = dataset.iloc[:,4].values
X[0:5]
```

```
array([[ 'Male', 19, 19000]],***
```

```
[36]: # onehot 编码
```

```
[37]: from sklearn.preprocessing import LabelEncoder, OneHotEncoder
from sklearn.compose import ColumnTransformer
labelencoder_X = LabelEncoder() # 先用LabelEncoder把分类变量标签变化，就是用数字标号代替文字
X[:, 0] = labelencoder_X.fit_transform(X[:, 0])
ct = ColumnTransformer(transformers=[('encoder', OneHotEncoder(), [0])], remainders='passthrough')
X = np.array(ct.fit_transform(X))
```

```
[38]: #from sklearn.preprocessing import LabelEncoder,OneHotEncoder
#labelencoder = LabelEncoder()
#X[:,0] = labelencoder.fit_transform(X[:,0])
#X[0:5]
#onehotencoder = OneHotEncoder(categorical_features=[0])
#X = onehotencoder.fit_transform(X).toarray()
#X[0:5]
```

```
[40]: from sklearn.model_selection import train_test_split
```

```
[41]: X_train,X_test,y_train,y_test = train_test_split(X,y,test_size = 0.2,random_state =0)
```

```
[42]: # 特征缩放
```

```
[43]: from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

```
[44]: # 训练
```

```
[45]: from sklearn.neighbors import KNeighborsClassifier
classifier = KNeighborsClassifier(n_neighbors=5,metric='minkowski',p=2)
classifier.fit(X_train,y_train)
```

```
<style>#sk-container-id-2 {color: black;}#sk-container-id-2 pre{padding: 0;}#sk-container-id-2 div.sk-toggleable {background-color: white;}#sk-container-
```

```
[46]: # 预测
```

```
[47]: y_pred = classifier.predict(X_test)
```

```
[48]: from sklearn.metrics import confusion_matrix
```

```
[49]: cm = confusion_matrix(y_test,y_pred)
cm
```

```
array([[55,  3],
       [ 1, 21]])
```

```
accuracy = (cm[0,0]+cm[1,1])/(cm[0,0]+cm[0,1]+cm[1,0]+cm[1,1])
```

```
[50]: 0.95
```

2-1 比较KNN与Logistic在这个数据案例上的结果差异，哪个算法对这个数据更有效；

KNN算法

生成混淆矩阵

- 我们怎么知道测试结果的好坏呢？
- 使用的工具，就是混淆矩阵。这个东西听起来很吓人，其实很简单
- 你想想看，我们的数据，或者客户，被分成了0或者1，如果“熟模型”，可以把测试集里面所有的0都正确判断为0，所有的1都正确判断为1，那么这个模型就非常厉害，它的准确率就是100%
- 但是，熟模型，作为一个机器，它是会犯错的。通俗的讲，它会糊涂，它会搞混淆，把本该是0判断成1，把本该是1判断成0
- 那么，它到底把多少0判断成1，把多少1判断成0，我们就用一个统计的工具“混淆矩阵”，来罗列出来
- 这就是混淆矩阵。就是判断模型的混淆程度，或者糊涂程度

```
[29]: from sklearn.metrics import confusion_matrix # 这个工具也是现成的
```

```
[31]: cm = confusion_matrix(y_test, y_pred) # y_test就是测试集原来的标签（人给的）；y_pred就是熟模型给的，他们两个的差异，就是混淆矩阵
```

- 就长下面这个样子
- 简单来说，对角线上的，就是全部判断对的，所有4个值加起来，就是测试数据总个数100
- 64对应的是 (0, 0)，0全部判断为0；4代表，有4个应该是0判断成1；3代表，有3个应该是1判断成0；29代表，1判断对了的个数
- 这样就是100个当中，7个判断错了，93个判断对了

```
[32]: cm
```

```
[32]: array([[55,  3],  
          [ 1, 21]])
```

- 关于混淆矩阵，更加详细的内容，我特别建议大家参考下面的博客，这样可以了解的清楚一点
- 博客地址：<https://www.jianshu.com/p/27228ad417d4>

```
[33]: accuracy = (cm[0,0]+cm[1,1])/(cm[0,0]+cm[0,1]+cm[1,0]+cm[1,1]) # 用对角线上的和，除以总数，就是准确率。准确率93%，还真不错呢！
```

```
[34]: accuracy
```

```
[34]: 0.95
```

Logistic算法

第4步：评估预测

我们预测了测试集。现在我们将评估逻辑回归模型是否正确的学习和理解。因此这个混淆矩阵将包含我们模型的正确和错误的预测。

生成混淆矩阵

```
[14]: from sklearn.metrics import confusion_matrix  
cm = confusion_matrix(y_test, y_pred)
```

```
[16]: [y_test, y_pred]
```

```
[16]: [array([0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1,  
          0, 1, 0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0,  
          1, 0, 0, 1, 0, 1, 1, 0, 0, 0, 0, 1, 1, 0, 0, 0, 1, 0, 0, 1, 0, 1, 0, 1,  
          0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 1, 1]),  
       array([0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1,  
          0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0,  
          1, 0, 0, 1, 0, 1, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1,  
          0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 1, 1])]
```

```
[18]: cm
```

```
[18]: array([[57,  1],  
          [ 5, 17]])
```

```
[19]: p = (cm[0,0]+cm[1,1])/(cm[0,0]+cm[1,1]+cm[0,1]+cm[1,0])
```

```
[19]: 0.925
```

对比分析

- 核心结论：KNN 模型的准确率（0.95）高于 Logistic 回归（0.925），在本数据案例中，**KNN 算法的分类效果更优。**
- 原因分析：（1）该数据的购买行为分布呈现非线性决策边界，Logistic 回归本质是线性模型，难以拟合这种复杂的分类边界；（2）KNN 基于样本距离进行局部投票，更能捕捉年龄、收入等特征与购买行为之间的非线性关系，因此泛化能力更强。

2-2训练集的比例，如何影响结果；

调整训练集比例

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# 1. 读取数据
dataset = pd.read_csv('Social_Network_Ads.csv')

# 2. 取特征 (Age + EstimatedSalary) 和标签
X = dataset.iloc[:, [2, 3]].values
y = dataset.iloc[:, 4].values

# 3. 遍历不同的测试集占比 (对应不同的训练集占比)
for test_size in [0.1, 0.2, 0.25, 0.3, 0.4]:
    # 划分训练集和测试集
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=test_size, random_state=0
    )
    # 特征缩放
    sc = StandardScaler()
    X_train = sc.fit_transform(X_train)
    X_test = sc.transform(X_test)
    # 训练KNN模型
    model = KNeighborsClassifier(n_neighbors=5)
    model.fit(X_train, y_train)
    # 预测与评估
    y_pred = model.predict(X_test)
    acc = accuracy_score(y_test, y_pred)
    # 输出训练集占比和准确率
    print(f"train_size = {1 - test_size} acc = {acc}")

train_size = 0.9 acc = 0.925
train_size = 0.8 acc = 0.95
train_size = 0.75 acc = 0.93
train_size = 0.7 acc = 0.9166666666666666
train_size = 0.6 acc = 0.89375
```

测试集比例	训练集比例	Acc
10%	90%	0.925
20%	80%	0.95
25%	75%	0.93
30%	70%	0.917
40%	60%	0.894

结果解读

- 1、整体变化：随着训练集占比的提高，acc呈现先升后降的规律。
- 2、最优划分比例：训练集占比 80%（测试集占比 20%）此时模型既获得了足够的样本进行训练，也有足够规模的测试集来稳定评估，acc达到 0.95，为本次实验中的最高值。

2-3超参数K如何影响结果，设置K=1、3、7、9，试一试；

调整K值

```
[57]: import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# 读取数据
dataset = pd.read_csv('Social_Network_Ads.csv')

# 取特征和标签
X = dataset.iloc[:, [2, 3]].values
y = dataset.iloc[:, 4].values

# 划分训练集和测试集
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=0
)

# 特征缩放
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)

# 遍历不同K值，测试模型效果
for k in [1, 3, 5, 7, 9]:
    model = KNeighborsClassifier(n_neighbors=k)
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    print(f"k = {k} acc = {accuracy_score(y_test, y_pred)}")

k = 1 acc = 0.8875
k = 3 acc = 0.95
k = 5 acc = 0.95
k = 7 acc = 0.95
k = 9 acc = 0.95
```

K值	Acc
1	0.8875
3	0.95
5	0.95
7	0.95
9	0.95

结果解读

- 1、K=1时模型表现最差:准确率仅为0.8875，是所有取值中最低的。
- 2、当K取3、5、7、9时，模型准确率均达到0.95，在本次实验中为最高值。说明本数据集的分类边界相对清晰，模型对K值在合理区间内的变化不敏感，稳定性较好。

2-4 什么情况下会出现过拟合。

1、K 值取 1（邻居数过小）时

K=1 时，模型仅根据距离最近的 1 个样本进行预测，会完全记住训练集中的噪声和异常点；例如数据中年龄、薪资的极端值会被模型当作“规律”学习，导致在测试集上泛化能力骤降，出现典型的过拟合现象。

2、训练集占比过高（如 90%），测试集过小时

- ①当训练样本过多、测试样本过少时，模型更容易记住训练集的细节而非通用规律；
- ②同时测试集样本量不足，无法有效评估模型的泛化能力，容易掩盖过拟合问题。

3、数据噪声大、异常点多，K 值过小时

本数据集中存在部分年龄、薪资的极端值，当 K 值过小时，这些异常点会被模型当作关键样本学习，导致决策边界扭曲，在测试集上表现不稳定。

3、对可视化图进行修改，同样是二维可视化的图，如果把性别变量考虑到可视化中

可视化代码

```
import pandas as pd
import matplotlib.pyplot as plt

# 读取数据
dataset = pd.read_csv('Social_Network_Ads.csv')
X = dataset.iloc[:, 1:4].values # 性别、年龄、薪资
y = dataset.iloc[:, 4].values # 是否购买

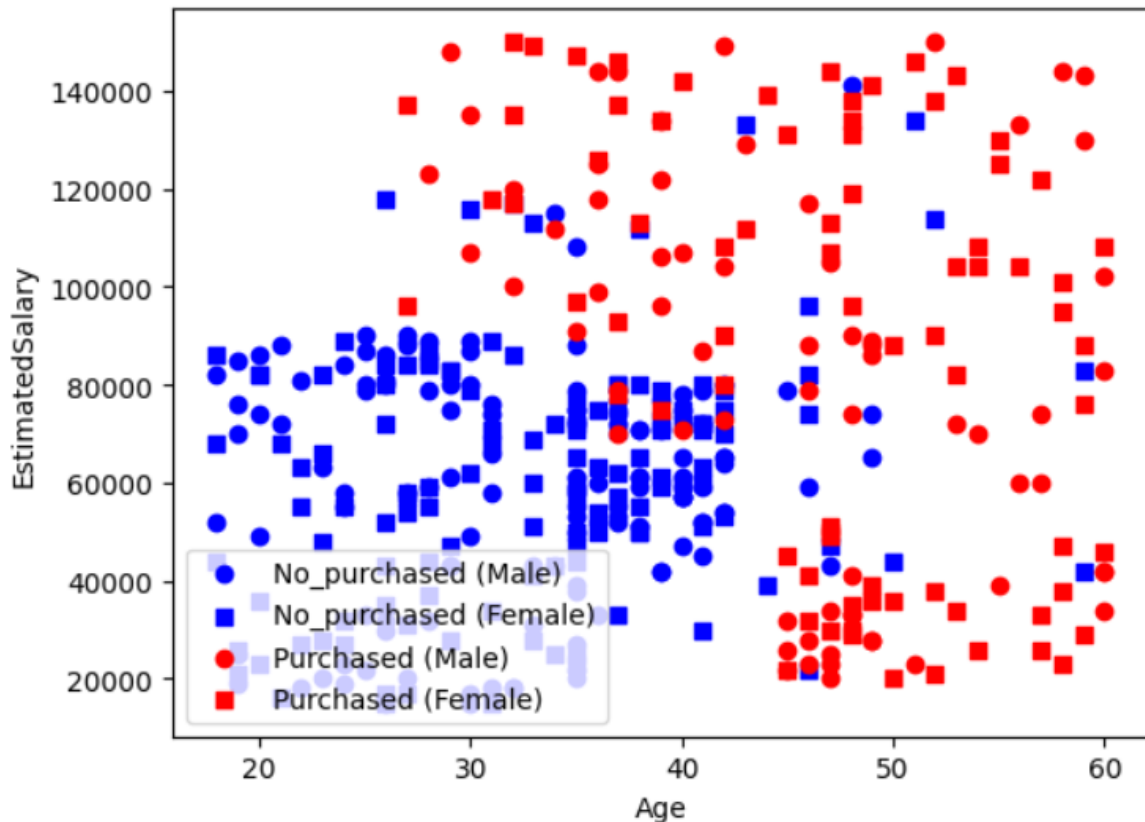
# 按是否购买拆分数据
x0 = X[y == 0] # 未购买
x1 = X[y == 1] # 已购买

# 进一步按性别拆分，实现：用颜色区分购买、用形状区分性别
# 未购买 (x0):
# - 男性: 蓝色圆圈
# - 女性: 蓝色方块
plt.scatter(x0[x0[:,0] == 'Male', 1], x0[x0[:,0] == 'Male', 2],
            c='blue', marker='o', label='No_purchased (Male)')
plt.scatter(x0[x0[:,0] == 'Female', 1], x0[x0[:,0] == 'Female', 2],
            c='blue', marker='s', label='No_purchased (Female)')

# 已购买 (x1):
# - 男性: 红色圆圈
# - 女性: 红色方块
plt.scatter(x1[x1[:,0] == 'Male', 1], x1[x1[:,0] == 'Male', 2],
            c='red', marker='o', label='Purchased (Male)')
plt.scatter(x1[x1[:,0] == 'Female', 1], x1[x1[:,0] == 'Female', 2],
            c='red', marker='s', label='Purchased (Female)')

plt.xlabel('Age')
plt.ylabel('EstimatedSalary')
plt.legend(loc=3)
plt.show()
```

可视化结果





决策树算法 课后作业9

 姓名：张苏蕊

 班级：信管2301

 学号：202321054007

1、加入性别因素，看看acc的变化

未添加性别因素

6: 预测

```
[11]: y_pred = clf.predict(X_test)

[12]: from sklearn.metrics import confusion_matrix

[13]: cm = confusion_matrix(y_test, y_pred)
      cm

[13]: array([[54,  4],
            [ 3, 19]])

[14]: accuracy = (cm[0,0]+cm[1,1])/(cm[0,0]+cm[0,1]+cm[1,0]+cm[1,1])
      accuracy

[14]: np.float64(0.9125)
```

添加性别因素后

```
: from sklearn.model_selection import train_test_split
  from sklearn.tree import DecisionTreeClassifier
  from sklearn.metrics import confusion_matrix

dataset = pd.read_csv('Social_Network_Ads.csv')
# 性别编码
dataset['Gender'] = dataset['Gender'].replace({'Male':1,'Female':0})
X = dataset.iloc[:, [1, 2, 3]].values
y = dataset.iloc[:, 4].values
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.25, random_state = 0)
classifier = DecisionTreeClassifier(random_state = 0)
classifier.fit(X_train, y_train)
y_pred = classifier.predict(X_test)
cm = confusion_matrix(y_test, y_pred)
acc = (cm[0,0]+cm[1,1])/len(y_test)
print("加入性别准确率: ",acc)

加入性别准确率:  0.91
```

结果分析

- 1、不加性别时，决策树模型的准确率为 0.9125；加入性别特征后，准确率为 0.91，两者仅存在极其微小的波动，模型性能没有发生实质性变化。
- 2、这说明在该数据集中，性别不是影响用户购买行为的关键因素。用户的购买决策主要由年龄和收入决定，性别对购买意愿的预测能力非常弱。

2、如何不进行特征缩放，预测的准确性有什么变化？

进行特征缩放

6: 预测

```
[11]: y_pred = clf.predict(X_test)

[12]: from sklearn.metrics import confusion_matrix

[13]: cm = confusion_matrix(y_test, y_pred)
      cm

[13]: array([[54,  4],
            [ 3, 19]])

[14]: accuracy = (cm[0,0]+cm[1,1])/(cm[0,0]+cm[0,1]+cm[1,0]+cm[1,1])
      accuracy

[14]: np.float64(0.9125)
```

不进行特征缩放

6: 预测

```
[11]: y_pred = clf.predict(X_test)

[12]: from sklearn.metrics import confusion_matrix

[13]: cm = confusion_matrix(y_test, y_pred)
      cm

[13]: array([[53,  5],
            [ 1, 21]])

[14]: accuracy = (cm[0,0]+cm[1,1])/(cm[0,0]+cm[0,1]+cm[1,0]+cm[1,1])
      accuracy

[14]: np.float64(0.925)
```

结果分析

- 1、数值对比：对决策树模型进行特征缩放时，准确率为 0.9125；不进行特征缩放时，准确率为 0.925。两者几乎没有变化，差异仅为 0.0125，属于正常微小波动。
- 2、核心结论：特征缩放对决策树模型的预测准确性几乎没有影响。决策树模型不需要特征缩放，是否缩放都不会明显改变分类效果。

3、比较KNN、Logistic、决策树、随机森林在该数据集上的表现。

概念阐释

一元线性回归: $y = b_0 + b_1 * x$

b0:截距 (直线与 y 轴的交点, $x=0$ 时的预测值)

b1:斜率 (系数, x 每增加 1, y 增加多少)

对应代码

```
b1 = regressor.coef_
```

输出: `array([9.91065648])` 即斜率约等于9.91。

```
b0 = regressor.intercept_
```

输出: `np.float64(2.0181600414346974)` 即截距约等于2.02。

最终模型: $y=2.02+9.91x$

代码解释

1、`regressor.coef_`

①存储的是所有特征的斜率 (系数) ;

②该案例中, 特征为1, 所以`coef_`是一个长度为1的数组 `[9.91065648]`, 即 b_1 。

③若为多元线性回归, `coef_`会是 `[b1, b2, ...]`, 对应多个特征的斜率。

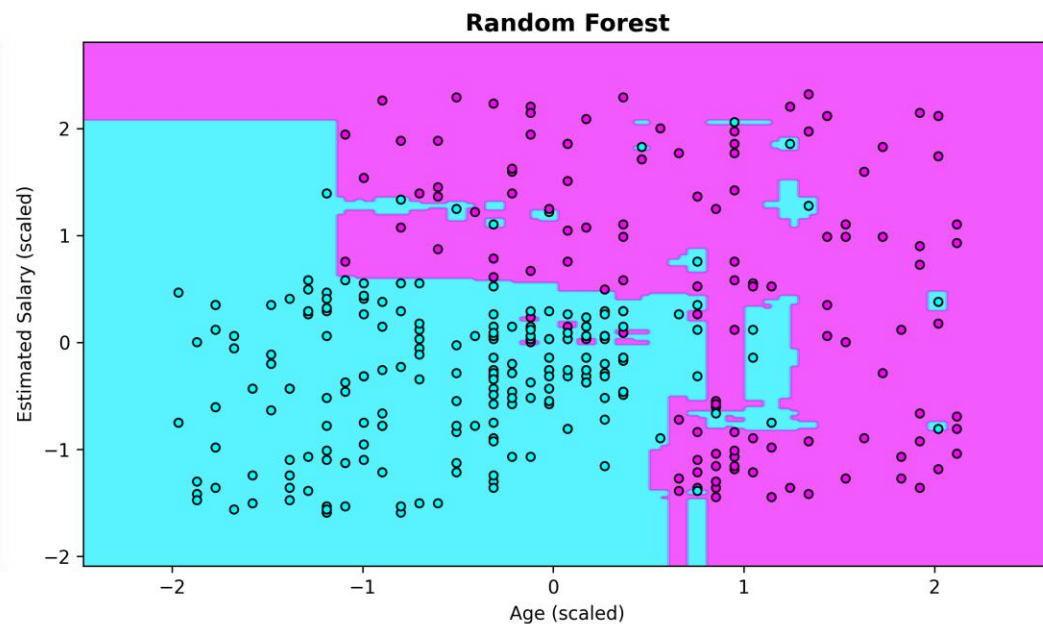
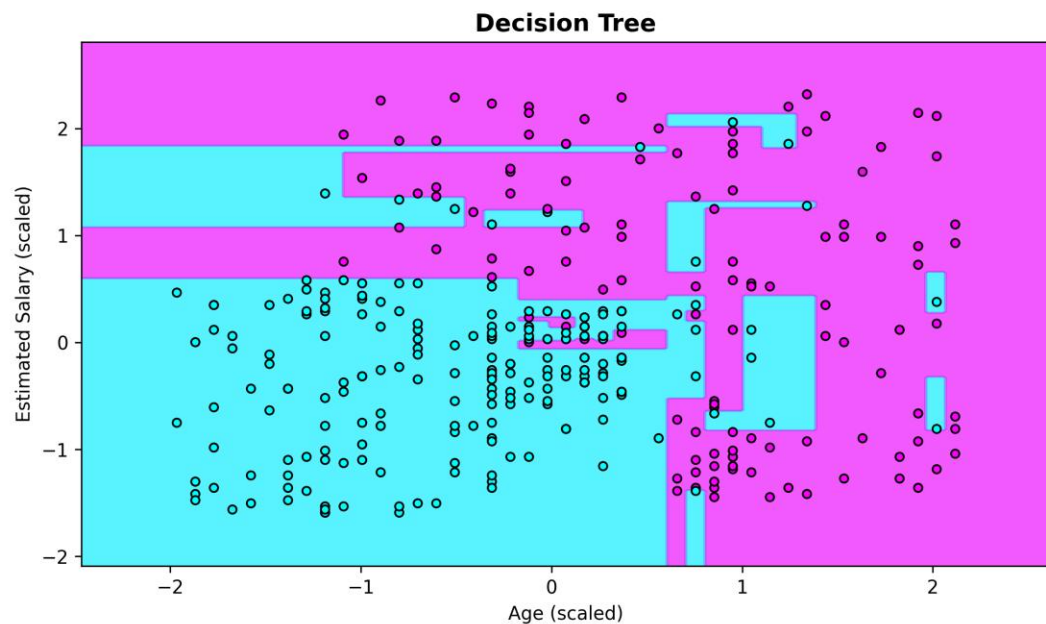
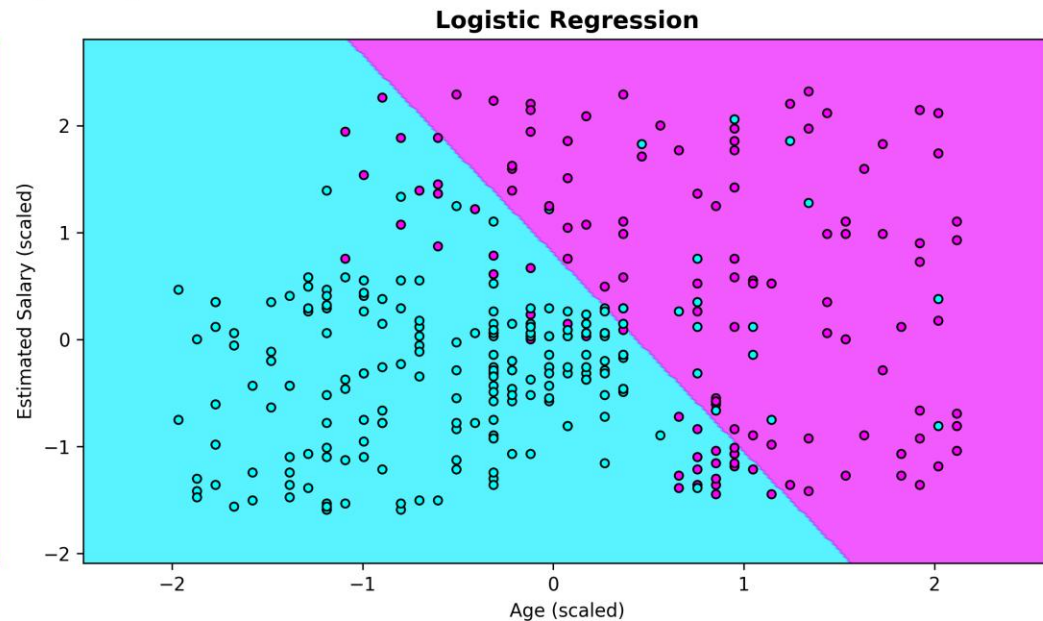
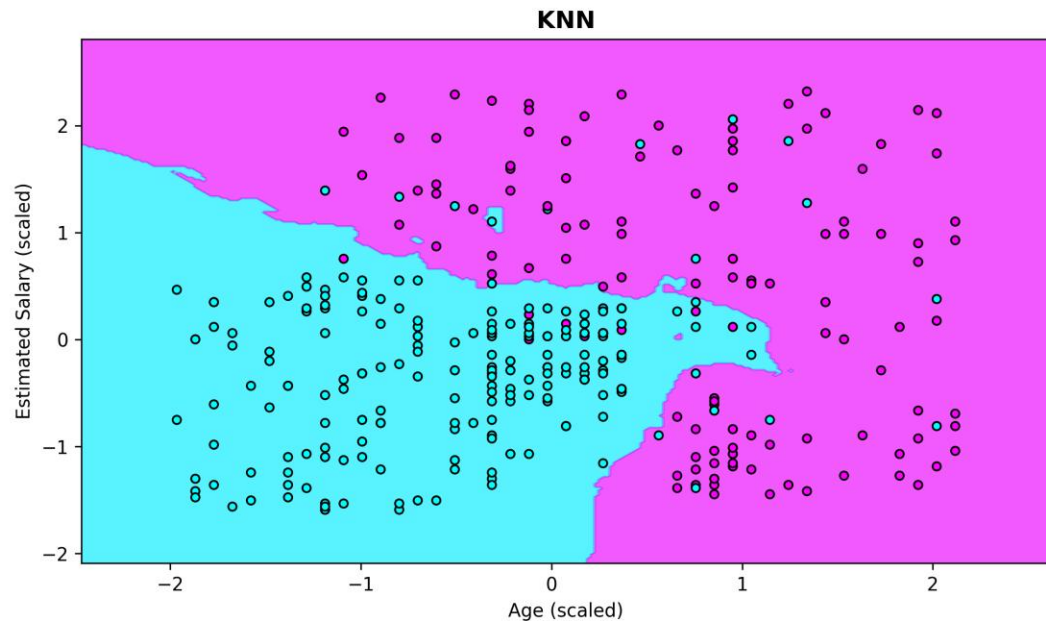
2、`regressor.intercept_`

①是模型自动计算的截距 b_0 , 是一个单独的浮点数 (不是数组) ;

②表示当 $x=0$ 时, y 的预测值。

3、比较KNN、Logistic、决策树、随机森林在该数据集上的表现。

Decision Boundary Comparison



3、比较KNN、Logistic、决策树、随机森林在该数据集上的表现。

各个算法比较

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import confusion_matrix

# 1. 导入数据
dataset = pd.read_csv('Social_Network_Ads.csv')
X = dataset.iloc[:, [2, 3]].values
y = dataset.iloc[:, 4].values

# 2. 划分训练集测试集
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=0
)

# 3. 特征缩放
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)

# 4. 定义模型
models = {
    "KNN": KNeighborsClassifier(),
    "逻辑回归": LogisticRegression(random_state=0),
    "决策树": DecisionTreeClassifier(criterion="entropy", random_state=0),
    "随机森林": RandomForestClassifier(criterion="entropy", random_state=0)
}

# 5. 训练并计算准确率
print("==== 问题3: 不同模型准确率对比 =====")
for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    cm = confusion_matrix(y_test, y_pred)
    acc = (cm[0,0] + cm[1,1]) / (cm[0,0]+cm[0,1]+cm[1,0]+cm[1,1])
    print(f"{name} 准确率: {acc:.4f}")
```

结果分析

算法	Acc
KNN	0.9500
逻辑回归	0.9250
决策树	0.9000
随机森林	0.9375

1、模型性能：在本数据集上，模型准确率从高到低为：KNN > 随机森林 > 逻辑回归 > 决策树。

2、核心结论：对于本案例数据，KNN 模型的预测效果最好，随机森林次之，决策树由于单棵树的局限性，泛化能力相对较弱。同时，特征缩放对KNN、逻辑回归这类对量纲敏感的模型有显著帮助，是提升这类模型性能的关键步骤。



100%

1、**核心特征：**年龄（Age）是第一分裂特征，其次是收入

2、**关键规则：**年轻用户购买意愿普遍偏低；高收入群体中，年龄越大购买倾向越强。

3、**模型表现：**分类边界清晰，但末端存在大量单样本节点，存在轻微过拟合风险，这也对应了单棵决策树准确率低于随机森林的实验结果。



聚类算法 课后作业10

 姓名：张苏芯

 班级：信管2301

 学号：202321054007

1、用花萼（第一0列和第二1列）的数据来聚类

代码运行

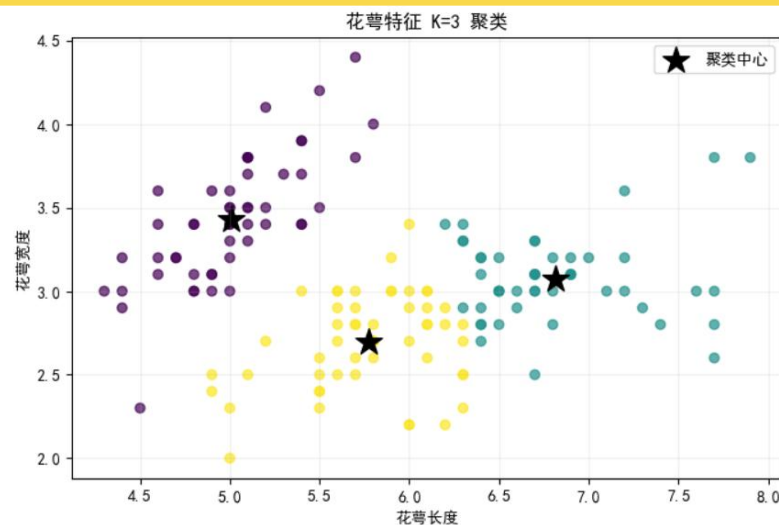
```
import matplotlib.pyplot as plt
import numpy as np
from sklearn.cluster import KMeans
from sklearn.datasets import load_iris

plt.rcParams["font.family"] = ["SimHei"]
plt.rcParams["axes.unicode_minus"] = False
iris = load_iris()
X = iris.data # 花萼长、花萼宽、花瓣长、花瓣宽

# 选取前两列花萼特征
X1 = iris.data[:, 0:2]
k1 = KMeans(n_clusters=3, n_init="auto", random_state=2025)
label1 = k1.fit_predict(X1)
center1 = k1.cluster_centers_

# 可视化
plt.figure(figsize=(8,5))
plt.scatter(X1[:,0], X1[:,1], c=label1, alpha=0.7)
plt.scatter(center1[:,0], center1[:,1], s=300, c="black", marker="*", label="聚类中心")
plt.xlabel("花萼长度")
plt.ylabel("花萼宽度")
plt.title("花萼特征 K=3 聚类")
plt.legend()
plt.grid(alpha=0.2)
plt.show()
```

结果分析



- (1) 紫色簇的样本分布较为紧凑，多围绕聚类中心聚集，簇内离散度低，说明该簇样本的花萼长度、宽度特征相似度高。
- (2) 黄色簇与青色簇的样本分布跨度较大，簇内部分样本远离形心，且两簇边界存在重叠，说明仅靠花萼二维特征，无法有效区分这两类样本的特征差异，簇内离散度高。

总结

聚类算法成功将样本划分为 3 个簇，实现了簇内样本聚集、簇间样本分离的效果，但受限于花萼特征的分度不足，仍有两类样本的划分边界不够清晰，聚类效果存在优化空间。

2、用所有数据来聚类（所有4列）

代码运行

```
# 全部特征
X2 = iris.data
k2 = KMeans(n_clusters=3, n_init="auto", random_state=2025)
label2 = k2.fit_predict(X2)
center2 = k2.cluster_centers_
```

输出关键数据

```
print("4个特征聚类")
print("各类簇样本数量：")
print(np.bincount(label2))
print("4个特征下聚类中心：")
print(center2)
print("当前整体簇内误差平方和：", k2.inertia_)
```

4个特征聚类

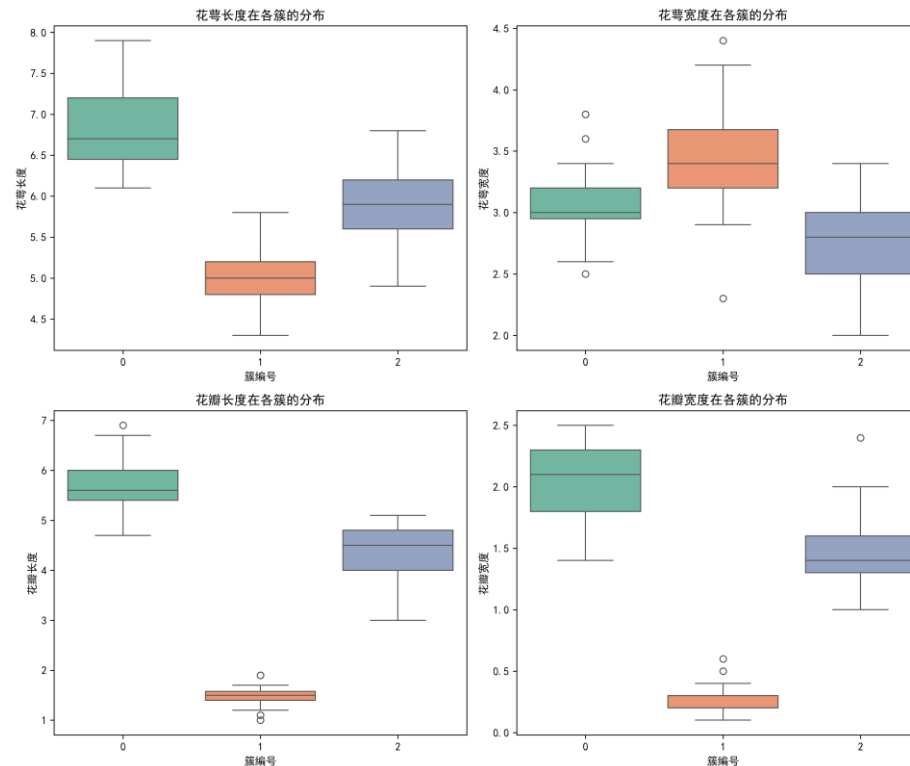
各类簇样本数量：

[50 39 61]

4个特征下聚类中心：

```
[[5.006      3.428      1.462      0.246      ]
 [6.85384615 3.07692308 5.71538462 2.05384615]
 [5.88360656 2.74098361 4.38852459 1.43442623]]
```

当前整体簇内误差平方和： 78.8556658259773



结果分析

1、样本分布：将样本划分3个簇，各簇的样本数量为[50, 39, 61]，分布相对均衡，无极端失衡的情况。

2、聚类中心：

- (1) 簇1：花瓣长度、宽度远小于其他两簇，是区分度最高的类别；
- (2) 簇2：花萼长度、花瓣尺寸均为最高值；
- (3) 簇3：特征数值介于两者之间，为中间尺寸类别。

特征分布箱线对比图

(1) 该图为各簇在 4 个特征上的分布箱线图，横轴为簇编号，纵轴为特征数值。箱体越短代表簇内样本越集中。

(2) 花瓣长度、宽度的箱线在 3 个簇中完全分离、无重叠，且箱体非常紧凑，说明花瓣是区分簇类的核心特征，簇内样本相似度极高；花萼特征的箱线虽有部分重叠，但分布区间仍有明显差异，起到了辅助划分的作用；

(3) 簇1（编号1）的所有特征分布最集中，几乎无离群点，聚类效果最稳定

3、这里的K=3，把K改成2，试一试

代码运行

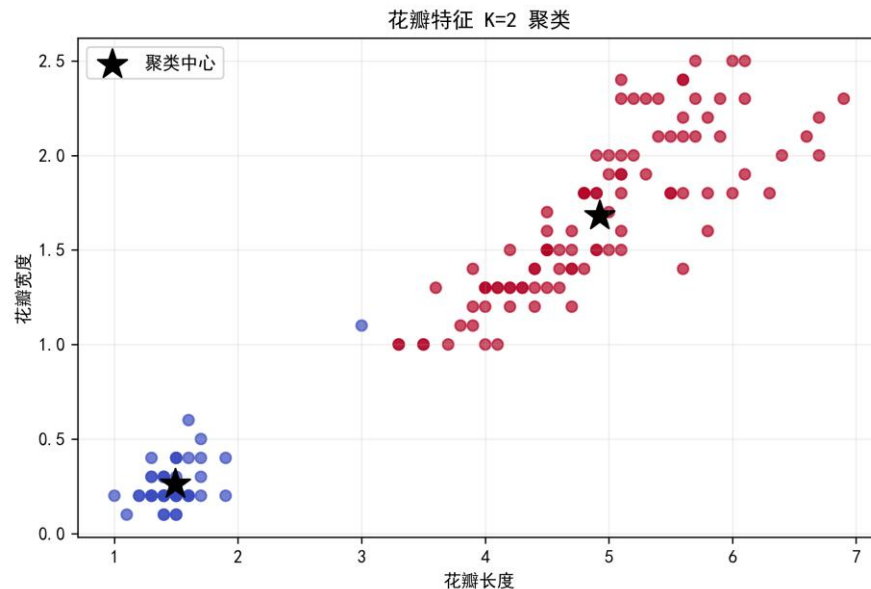
```
X3 = iris.data[:, 2:4]
k3 = KMeans(n_clusters=2, n_init="auto", random_state=2025)
label3 = k3.fit_predict(X3)
center3 = k3.cluster_centers_

plt.figure(figsize=(8,5))
plt.scatter(X3[:,0], X3[:,1], c=label3, alpha=0.7, cmap="coolwarm")
plt.scatter(center3[:,0], center3[:,1], s=300, c="black", marker="*", label="聚类中心")
plt.xlabel("花瓣长度")
plt.ylabel("花瓣宽度")
plt.title("花瓣特征 K=2 聚类")
plt.legend()
plt.grid(alpha=0.2)
plt.savefig("cluster3.png", dpi=300, bbox_inches="tight")
plt.show()
```

K=2 与 K=3 聚类效果对比分析（针对花瓣特征）

- 1、K=2时聚类效果：边界清晰、划分稳定
- 2、K=3时聚类效果：划分边界模糊、簇间易重叠
- 3、针对花瓣特征来看，K=2时聚类算法划分与样本的数据特征较为契合，簇间完全分离、簇内一致性高；而K=3会存在两个簇划分边界模糊，聚类质量下降。这也体现了 K-Means 算法中选择合适的 K 值的重要性。

结果分析



聚类划分效果

- (1) 蓝色簇：样本集中在花瓣长度1-2、宽度0-0.6的区域，样本分布较为紧凑。
- (2) 红色簇：样本集中在花瓣长度3-7、宽度1.0-2.5的区域，主要围绕右上角的聚类中心分布。
- (3) 两个簇的样本分布体现出花瓣特征的区分度高，K=2时划分边界非常清晰。

4、聚类的其他输出，可视化聚类中心。

代码运行

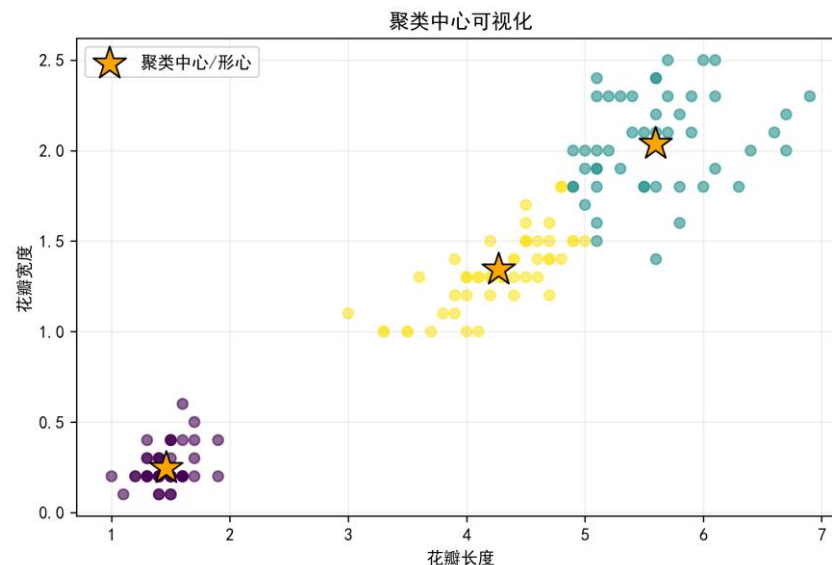
```
X4 = iris.data[:, 2:4]
k4 = KMeans(n_clusters=3, n_init="auto", random_state=2025)
k4.fit(X4)
center4 = k4.cluster_centers_
print("聚类中心(形心)")
print(center4)

plt.figure(figsize=(8,5))
plt.scatter(X4[:,0], X4[:,1], c=k4.labels_, alpha=0.6)
plt.scatter(center4[:,0], center4[:,1], s=400,
            c="orange", marker="*", edgecolors="black", label="聚类中心/形心")
plt.xlabel("花瓣长度")
plt.ylabel("花瓣宽度")
plt.title("聚类中心可视化")
plt.legend()
plt.grid(alpha=0.2)
plt.savefig("center4.png", dpi=300, bbox_inches="tight")
plt.show()
```

聚类中心(形心)

```
[[1.462      0.246    ]
 [5.59583333 2.0375    ]
 [4.26923077 1.34230769]]
```

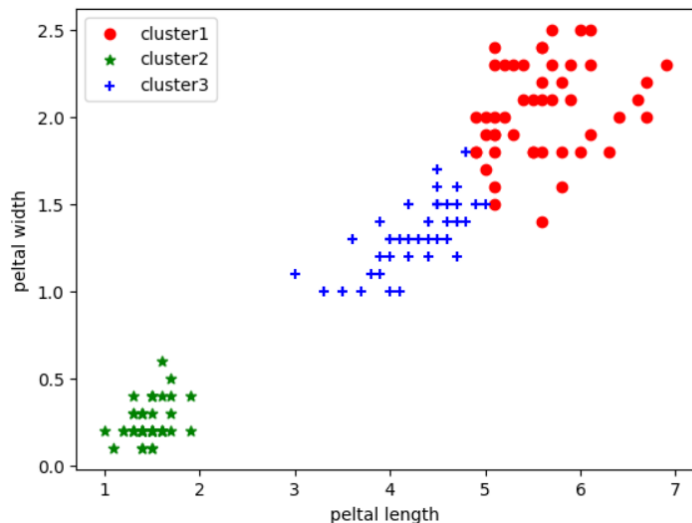
结果分析



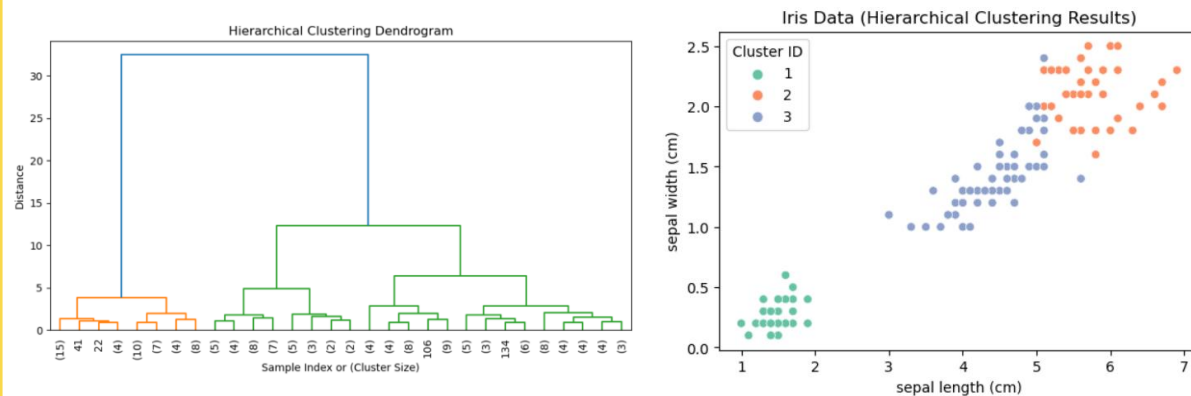
- 1、聚类中心是每个簇所有样本的平均代表，是K-Means算法迭代优化的核心，可以帮助我们快速识别每个簇的“典型特征”。
- 2、从图中可以看到，每个簇的样本都紧密地围绕着自己的聚类中心分布，说明簇内样本的相似度高。
- 3、聚类中心之间的距离：三个聚类中心在图上彼此分离，距离较远，这说明三个簇之间差异显著。

5、针对该案例K-Means算法与HC算法对比分析

K-Means算法



HC算法



维度	K-Means	HC
核心原理	基于距离的迭代优化，通过不断更新聚类中心，最小化簇内平方误差和，实现各个簇的分组。	基于层级合并/分裂，以树状结构来展示样本间的相似性，通过合并最相似的簇，构建完整的聚类层级。
优势	聚类中心可代表簇的典型特征，结果清晰易解释	树状图可展示样本的层次关系，清晰直观
局限	会因为部分异常点拉偏聚类中心，增加误差	计算复杂，大规模数据时所呈现的树状图可读性差
本案例效果	中等花瓣与大花瓣簇的边界存在明显重叠，部分样本特征值介于两者之间	从树状图来看，三个簇的分布区间在花瓣特征上较为清晰，无重叠